

ВВЕДЕНИЕ

Сервисы заказа такси прочно вошли в повседневную жизнь современного города. Пользователи всё реже ловят автомобиль на улице или звонят диспетчеру, предпочитая оформлять заказ через мобильное приложение. Это требует от серверной части таких сервисов высокой производительности, отказоустойчивости и способности обрабатывать большое количество одновременных запросов.

Данный дипломный проект посвящён разработке серверной части приложения заказа такси. Серверная часть является ключевым компонентом любой подобной платформы: именно на сервере выполняется обработка заказов, поиск свободных водителей, расчёт стоимости поездки, хранение данных о пользователях и координация взаимодействия между всеми участниками процесса.

От качества реализации серверной части напрямую зависит стабильность работы всего приложения. Недостаточная производительность или частые сбои приводят к потере пользователей и снижению конкурентоспособности сервиса. Поэтому проектированию и реализации серверной части должно уделяться особое внимание.

В рамках данной работы разработана серверная часть на основе микросервисной архитектуры. Каждый функциональный блок системы вынесен в отдельный сервис со своей базой данных, а взаимодействие между ними организовано через REST API и брокер сообщений. Такой подход обеспечивает гибкость, независимое развёртывание компонентов и возможность горизонтального масштабирования.

В первой главе проведён анализ предметной области и сформулированы требования к системе. Вторая глава содержит обзор технологических решений и обоснование выбранного стека. В третьей главе спроектирована архитектура системы. Четвёртая глава описывает реализацию ключевых модулей. Пятая глава посвящена тестированию и развёртыванию. В шестой главе выполнен расчёт экономической эффективности.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Общие сведения о сервисах заказа такси

Сервисы заказа такси представляют собой цифровые платформы, обеспечивающие взаимодействие пассажиров, водителей и операторов сервиса в рамках единого информационного пространства. Основная задача такого сервиса заключается в быстром приеме заказа, определении подходящего исполнителя, построении маршрута поездки, расчете стоимости и сопровождении заказа на всех стадиях его жизненного цикла.

Современный рынок такси существенно отличается от классической модели диспетчерской службы. Если ранее ключевая роль отводилась оператору, который вручную принимал звонки и передавал заявку водителю, то теперь значительная часть операций автоматизирована. Пользователь самостоятельно формирует заказ через мобильное приложение, указывает точку подачи автомобиля, пункт назначения, способ оплаты и дополнительные пожелания. После этого серверная часть системы выполняет обработку запроса и инициирует дальнейшие бизнес-процессы [4].

В рамках предметной области можно выделить несколько основных участников. Пассажир создает заказ, отслеживает подачу автомобиля, получает информацию о водителе и оплачивает поездку. Водитель принимает или отклоняет предложенный заказ, прибывает в точку подачи, выполняет перевозку и завершает поездку. Администратор сервиса управляет тарифами, пользователями, промокампаниями, справочниками, зонами обслуживания и механизмами контроля качества. Помимо этого, в системе могут участвовать внешние сервисы: платежные шлюзы, геокодеры, картографические платформы, сервисы уведомлений и сообщений.

Заказ такси как сущность предметной области обладает набором состояний. В типичном сценарии заказ последовательно проходит стадии "создан" "назначается" "принят водителем" "водитель прибыл" "поездка начата" "поездка завершена" "оплачен" либо "отменен". Изменение состояния сопровождается проверками бизнес-правил, обновлением данных в нескольких подсистемах и уведомлением заинтересованных участников. Следовательно, серверная часть приложения должна обеспечивать не только хранение состояния, но и строгую координацию переходов между этими состояниями [1].

Существенной особенностью сервисов заказа такси является их географическая привязка. Для работы платформы необходимо постоянно обрабатывать координаты пассажиров и водителей, рассчитывать расстояния, оценивать примерное время подачи автомобиля и прогнозировать продолжительность поездки. Данные задачи требуют интеграции с картографическими системами и алгоритмами маршрутизации. При этом серверная часть должна работать с координатами в режиме, близком к реальному времени, так как даже небольшая задержка может привести к неверному выбору водителя или ухудшению пользовательского опыта [6,7].

Важной характеристикой данной предметной области является и зависимость качества сервиса от скорости обмена данными между клиентской и серверной частями. Пассажир ожидает быстрого подтверждения заказа и получения достоверной информации о подаче автомобиля. Водителю, в свою очередь, необходим своевременный доступ к сведениям о новом заказе, маршруте и статусе поездки. Следовательно, серверная часть должна

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		6

обеспечивать непрерывный обмен данными и согласованное представление состояния заказа для всех участников процесса.

Еще одной характерной чертой является высокая событийность предметной области. В течение короткого промежутка времени в системе могут происходить десятки тысяч однотипных действий: создание заказов, обновление геопозиции, смена статусов поездок, расчет стоимости, отправка уведомлений, авторизация пользователей, фиксация платежных операций. Эти события возникают асинхронно, пересекаются по времени и затрагивают разные части системы. По этой причине серверная часть должна проектироваться как основа для обработки большого числа конкурентных запросов и событий [2].

Бизнес-логика приложений такси не сводится только к поиску ближайшего автомобиля. На практике сервер должен поддерживать тарифные планы, динамическое ценообразование, валидацию промокодов, привязку банковских карт, историю поездок, рейтинговую систему, блокировки пользователей, обработку жалоб, возвраты средств и формирование аналитики. Дополнительно возникают задачи, связанные с безопасностью поездки, подтверждением личности водителя, хранением документов и журналированием действий пользователей. Таким образом, предметная область сочетает в себе элементы транспортной логистики, финансовых расчетов, геоинформационной обработки и управления пользовательскими аккаунтами.

С точки зрения программной реализации сервис заказа такси обычно включает несколько клиентских приложений и единую серверную платформу. К клиентским компонентам относятся мобильное приложение пассажира, мобильное приложение водителя, веб-панель администратора, а иногда и интерфейс операторов контакт-центра. Серверная часть выступает центральным звеном ИС, поскольку именно она реализует доменную логику, управляет доступом к данным, координирует взаимодействие между подсистемами и обеспечивает интеграцию с внешними платформами.

Типичный процесс выполнения заказа можно представить следующим образом. Пользователь проходит аутентификацию, отправляет запрос на создание поездки, после чего сервер валидирует входные данные и рассчитывает предварительную стоимость. Далее подсистема распределения заказов определяет список доступных водителей в заданном радиусе, выполняет ранжирование кандидатов и инициирует механизм назначения. После подтверждения заказа водителем система переводит заказ в активное состояние, сохраняет события маршрута, фиксирует время прибытия и поддерживает обмен статусами между участниками. По завершении поездки выполняется окончательный расчет стоимости, списание средств, обновление балансов и запись информации в историю пользователя [5].

На практике отдельные этапы жизненного цикла заказа могут различаться в зависимости от бизнес-правил конкретного сервиса. Например, может использоваться предварительное резервирование суммы поездки, автоматическое назначение водителя без подтверждения либо ручная отмена поездки со стороны оператора. Несмотря на различия в деталях реализации, базовая логика остается общей: система должна корректно принимать заказ, назначать исполнителя, сопровождать поездку и фиксировать результат выполнения услуги.

Серверная часть занимает центральное место в структуре программного комплекса. Она обеспечивает взаимодействие клиентских приложений с подсистемами авторизации, обработки заказов, геолокации, уведомлений и хранения данных. Такое положение определяет ее роль как координирующего элемента всей ИС и задает требования к производительности, сопровождаемости и возможности дальнейшего расширения.

Анализ предметной области показывает, что серверная часть приложения такси является критически важным компонентом всей платформы. Именно от ее архитектуры зависят скорость обработки заказов, корректность бизнес-операций и удобство дальнейшего развития продукта. Следовательно, разработка серверной части должна учитывать как специфику бизнес-процессов такси, так и технические ограничения распределенных систем, даже если на текущем этапе реализуется минимально жизнеспособный продукт.

1.2 Особенности разработки высоконагруженных серверных систем

Высоконагруженной серверной системой принято считать программную систему, которая должна стабильно функционировать при большом количестве одновременных запросов, интенсивном обмене данными и строгих требованиях к времени отклика [2]. Для приложений заказа такси такая характеристика является естественной, поскольку нагрузка в системе распределена неравномерно и может резко возрастать в часы пик, во время неблагоприятных погодных условий, в праздничные дни и при проведении массовых мероприятий.

Одной из ключевых проблем разработки высоконагруженной системы является обеспечение горизонтальной масштабируемости. Если сервис строится таким образом, что увеличение числа пользователей требует только наращивания ресурсов одного сервера, то со временем возникает аппаратный и организационный предел. Более практичным подходом является проектирование компонентов, которые можно тиражировать на несколько экземпляров и распределять между ними входящий поток запросов. Для серверной части приложения такси это особенно важно, так как подсистемы аутентификации, управления заказами, работы с геолокацией и уведомлениями могут испытывать различный профиль нагрузки [2].

Существенное значение имеет производительность операций чтения и записи. В системе такси постоянно обновляются координаты водителей, создаются новые заказы, фиксируются изменения статусов и выполняются платежные транзакции. При этом часть данных должна быть доступна почти мгновенно, например текущие координаты автомобилей или состояние активного заказа. Другие данные могут обрабатываться с большей задержкой, например статистические отчеты и архивные журналы. Следовательно, при проектировании необходимо разделять горячие и холодные данные, использовать кэширование и выбирать подходящие модели хранения, включая БД и СУБД, соответствующие профилю конкретного сервиса [2,6].

Отдельной задачей является обеспечение отказоустойчивости. Сервис такси работает непрерывно, поэтому недоступность даже одной критической подсистемы может привести к массовым сбоям: пассажиры не смогут вызвать автомобиль, водители не получат заказы, а администрация не сможет контролировать состояние платформы. Для снижения подобных рисков применяются резервирование инстансов, балансировка нагрузки, репликация баз данных, очереди сообщений, механизмы повторной доставки событий и стратегии деградации функциональности [2,8].

Даже если на текущем этапе реализуется MVP, учет подобных факторов остается необходимым уже на стадии проектирования. Практика показывает, что игнорирование

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		8

вопросов устойчивости на раннем этапе приводит к существенным трудностям при дальнейшем развитии системы. По этой причине архитектурные решения должны изначально учитывать возможность отказа отдельных компонентов и предусматривать хотя бы базовые механизмы обработки нештатных ситуаций.

В высоконагруженных системах большое внимание уделяется согласованности данных. В серверной части приложения такси одна бизнес-операция может затрагивать несколько сущностей одновременно: заказ, пользователя, водителя, платежную запись, бонусный счет и журнал событий. Если на уровне архитектуры не предусмотрены механизмы координации, то система может попасть в противоречивое состояние. Например, заказ может быть отмечен как завершенный, но платеж не будет зафиксирован, либо несколько водителей получают один и тот же заказ. Поэтому необходимо заранее определить, где требуется строгая согласованность, а где допустима временная рассогласованность данных [2].

Высокая нагрузка напрямую связана с конкуренцией за ресурсы. Несколько потоков обработки могут одновременно изменять одни и те же записи, например при назначении водителя на заказ или при перерасчете стоимости поездки. Если такие ситуации не контролировать, возникают гонки данных, двойные списания средств, повторная отправка уведомлений и другие ошибки, которые негативно влияют на бизнес-процессы. Для предотвращения подобных проблем применяются оптимистические и пессимистические блокировки, идемпотентность запросов, уникальные ограничения на уровне БД и использование брокеров сообщений для упорядочивания отдельных операций [2,8].

Важной характеристикой высоконагруженной системы является наблюдаемость. Разработчику недостаточно просто развернуть сервис и убедиться, что он принимает запросы. Необходимо собирать метрики производительности, журналы событий, трассировки распределенных запросов и показатели ошибок. В приложении такси это позволяет обнаруживать узкие места, анализировать задержки в обработке заказов, отслеживать сбои внешних интеграций и оперативно реагировать на деградацию системы [2,3].

Наблюдаемость важна не только для промышленной эксплуатации, но и для этапа разработки. Наличие журналов и базовых метрик позволяет выявлять ошибки бизнес-логики, анализировать последовательность переходов заказа между состояниями и контролировать корректность межсервисного взаимодействия. Следовательно, средства логирования и мониторинга следует рассматривать как составную часть архитектуры, а не как дополнительный элемент, вводимый после завершения основной реализации.

Особое внимание должно уделяться безопасности. Серверная часть обрабатывает персональные данные пользователей, сведения о поездках, контактную информацию водителей, документы, платежные токены и историю операций. Соответственно, необходимы аутентификация и авторизация, защита каналов передачи данных, разграничение прав доступа, аудит критичных действий и механизм безопасного хранения секретов. Для внешних API также должны быть предусмотрены ограничение частоты запросов, защита от автоматизированных атак и валидация входных данных. В качестве одного из практических механизмов авторизации могут использоваться JWT [4,5].

Еще одна особенность высоконагруженных систем состоит в том, что не все операции должны выполняться синхронно. Некоторые действия допустимо выносить в асинхронную обработку, если это не ухудшает пользовательский сценарий. К таким операциям относятся отправка чеков, уведомлений, построение аналитических витрин, генерация отчетов, индексирование данных для поиска и часть интеграционных обменов. Использование очередей сообщений и событийного взаимодействия снижает пиковую нагрузку на центральные компоненты и делает систему более устойчивой к временным сбоям [8].

Для предметной области такси критически важна также работа со временем отклика. Пользователь ожидает, что приложение покажет стоимость поездки за короткий интервал времени, а поиск водителя завершится без заметной задержки. Слишком долгая обработка воспринимается как ошибка сервиса и приводит к потере заказов. Поэтому архитектурные решения должны оцениваться не только по функциональной полноте, но и по их влиянию на скорость обработки запросов [2].

Таким образом, разработка высоконагруженной серверной системы для приложения такси требует учета масштабируемости, отказоустойчивости, согласованности данных, производительности, безопасности и наблюдаемости. Эти требования непосредственно влияют на выбор архитектурного стиля и объясняют необходимость анализа различных подходов к организации серверной части [2].

1.3 Сравнительный анализ монолитной и микросервисной архитектур

Архитектура серверного приложения определяет способ разбиения системы на логические части, характер их взаимодействия, принципы развертывания и сопровождения. Для разработки серверной части приложения такси наиболее показательным является сравнение двух широко используемых подходов: монолитной и микросервисной архитектур [1,4].

Монолитная архитектура предполагает создание единого приложения, в котором все основные функции системы реализованы в рамках общего кода, единого процесса выполнения и, как правило, общей базы данных. В таком приложении модули могут быть логически разделены по пакетам и слоям, однако с точки зрения развертывания и масштабирования они остаются частью одного исполняемого сервиса [1,4].

Преимущество монолитного подхода состоит в относительной простоте начальной разработки. Единый проект легче собрать, протестировать и запустить, особенно на ранней стадии, когда доменная область еще уточняется. Внутренние вызовы между модулями выполняются без сетевых затрат, транзакции проще реализуются в пределах одного процесса, а требования к инфраструктуре и сопровождению обычно ниже. Для небольших систем с ограниченной функциональностью монолит может быть рациональным выбором.

Однако по мере роста приложения монолитная архитектура начинает проявлять существенные ограничения. Усложняется кодовая база, увеличивается связность между модулями, становится труднее изолировать изменения и безопасно выпускать новые версии. Масштабирование выполняется целиком для всего приложения, даже если повышенная нагрузка характерна только для одной части, например для модуля геолокации или обработки заказов. Ошибка в одном компоненте способна повлиять на доступность всей системы, а длительное время сборки и тестирования замедляет развитие продукта [1,2].

Микросервисная архитектура предлагает иной подход, при котором система разбивается на набор относительно независимых сервисов, каждый из которых реализует ограниченную бизнес-область, владеет своей логикой и обычно имеет собственное хранилище данных [1]. Такие сервисы взаимодействуют через сетевые протоколы, например HTTP, REST API или брокер сообщений, а развертывание и масштабирование могут вы-

полняться независимо друг от друга.

Для предметной области такси микросервисный подход выглядит естественным, поскольку бизнес-функции достаточно отчетливо разделяются. Отдельно можно выделить сервис пользователей, сервис водителей, сервис заказов, сервис тарифов, сервис платежей, сервис уведомлений, сервис геолокации и сервис аналитики. Каждый из них имеет собственный жизненный цикл, собственные требования к производительности и разные внешние интеграции. Такое разделение способствует более четкому управлению ответственностью и снижает связанность компонентов [1].

К преимуществам микросервисной архитектуры относится возможность независимого масштабирования. Если в системе возрастает число запросов на расчет маршрутов и подбор водителей, можно увеличить число экземпляров только соответствующих сервисов, не затрагивая, например, административный модуль или сервис истории поездок. Кроме того, становится проще локализовать сбои, так как отказ одного сервиса не обязательно приводит к полной недоступности всей платформы. При корректной настройке механизмов деградации и повторов система может продолжить работу в частично ограниченном режиме [1,2].

Еще одно важное преимущество микросервисов заключается в технологической и организационной гибкости. Для Java-проектов этот подход особенно удобен благодаря развитой экосистеме средств разработки и развертывания, включая JDK, Spring Boot и REST API [3,5]. В рамках данной работы это важно, так как проект реализуется на языке Java и должен опираться на зрелые технологические решения.

Вместе с тем микросервисная архитектура влечет за собой новые сложности. Межсервисное взаимодействие происходит по сети, а значит появляются задержки, сетевые ошибки, необходимость версионирования API и контроля форматов сообщений. Транзакции между несколькими сервисами становятся сложнее, так как невозможно без дополнительных механизмов использовать привычную модель единой локальной транзакции. Возрастает роль централизованного логирования, трассировки, управления конфигурацией и балансировки нагрузки [1].

Кроме того, микросервисный подход требует более тщательного проектирования границ ответственности. Если границы сервисов выбраны неудачно, система может получить избыточное число сетевых вызовов, дублирование данных и чрезмерно сложные сценарии согласования изменений. Поэтому при декомпозиции серверной части необходимо стремиться к такому разделению, при котором каждый сервис решает ограниченный набор задач и обладает четко определенным набором данных и операций.

Существенным недостатком микросервисного подхода является повышенная сложность эксплуатации. Для стабильной работы распределенной системы требуется развитая инфраструктура развертывания, контейнеризация, автоматизация сборки и поставки, централизованный сбор метрик и продуманная политика отказоустойчивости. Если эти вопросы не решены, система может оказаться сложнее в сопровождении, чем монолит аналогичной функциональности. Следовательно, микросервисы оправданы не сами по себе, а тогда, когда их преимущества действительно покрывают инфраструктурные издержки [1,3].

Сравнение архитектурных подходов показывает, что монолитная архитектура обеспечивает более простую начальную реализацию, тогда как микросервисная архитектура лучше соответствует требованиям масштабирования, изоляции отказов и независимого развития функциональных компонентов. Для предметной области такси данные свойства сохраняют значимость, однако на раннем этапе разработки необходимо учитывать баланс

между архитектурной гибкостью и сложностью реализации.

Для разрабатываемой серверной части приложения такси микросервисная архитектура рассматривается как предпочтительное направление проектирования. Такой выбор обусловлен несколькими причинами. Во-первых, предметная область естественным образом разбивается на относительно автономные контексты. Во-вторых, даже минимально жизнеспособный продукт целесообразно строить с учетом возможного роста нагрузки на отдельные сервисы. В-третьих, выбранный подход создает основу для последующего расширения функциональности без полной переработки архитектуры. Наконец, использование Java и экосистемы современных фреймворков позволяет реализовать такой подход в рамках технологически обоснованного решения [1,3].

При этом выбор микросервисной архитектуры не отменяет необходимости соблюдения инженерной дисциплины [4]. Поэтому сервисы должны быть спроектированы с четкими границами ответственности, а межсервисные контракты - оставаться стабильными и минимально связанными.

1.4 Требования к серверной части приложения и постановка задачи

На основании рассмотренной предметной области и особенностей высоконагруженных систем можно сформулировать требования к серверной части приложения такси. Эти требования определяют состав разрабатываемого решения, его архитектурные ограничения и критерии качества. При этом практическая реализация в рамках данной работы ориентирована на минимально жизнеспособный продукт, покрывающий ключевые пользовательские сценарии.

Прежде всего серверная часть должна обеспечивать базовые функциональные возможности. К ним относятся регистрация и аутентификация пользователей, хранение профилей пассажиров и водителей, создание заказа, назначение водителя на поездку, сопровождение заказа по основным состояниям, расчет предварительной стоимости, обработка отмен и предоставление пользователю актуального статуса поездки [5].

Указанный набор функций следует рассматривать как минимально необходимый для демонстрации работоспособности системы. Реализация именно этих сценариев позволяет подтвердить корректность выбранной архитектуры, проверить взаимодействие между ключевыми сервисами и обеспечить заверченный пользовательский процесс от создания заказа до получения результата. Функции, связанные с бонусными программами, расширенной аналитикой и сложными финансовыми операциями, могут быть отнесены к следующим этапам развития продукта.

Для корректной работы в предметной области такси система должна поддерживать работу с геоданными. Это включает прием координат от мобильных клиентов, хранение актуального местоположения водителей, поиск ближайших исполнителей, взаимодействие с картографическими сервисами и передачу пользователям информации о маршруте и времени подачи автомобиля. Следовательно, серверная часть должна предоставлять API для оперативного обмена географическими данными и учитывать особенности их высокой изменчивости [6,7].

Существенным функциональным требованием является поддержка ролевой мо-

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		12

дели доступа. Пассажир, водитель и администратор работают с разными сценариями, поэтому сервер обязан разграничивать права доступа и проверять полномочия на уровне каждого защищенного ресурса. Отдельно следует предусмотреть управление административными операциями, связанными с тарифами, пользователями, справочниками и мониторингом состояния системы, а также защиту соответствующих API [4,5].

К числу нефункциональных требований относится производительность. Серверная часть должна обеспечивать приемлемое время отклика для основных пользовательских операций. Наиболее чувствительными сценариями являются аутентификация, создание заказа, поиск водителя и получение текущего статуса поездки [2].

Другим важным нефункциональным требованием является масштабируемость. Так как количество пользователей и интенсивность заказов могут изменяться во времени, проектируемая система должна допускать развитие без полной переработки архитектуры. Это означает необходимость выделения независимых сервисов и использования решений, которые позволяют в дальнейшем масштабировать отдельные компоненты по мере роста нагрузки [1,2].

Серверная часть также должна учитывать требования устойчивости к ошибкам. Даже в минимально жизнеспособном продукте необходимо предусмотреть обработку ошибок внешних интеграций, повторные попытки выполнения отдельных операций и сохранение критически важных данных. Наибольшее значение эти меры имеют для сервисов, связанных с заказами и авторизацией [2,8].

Требования безопасности включают защищенную аутентификацию, хранение паролей в безопасном виде, использование токенов доступа, валидацию входных данных, ограничение частоты запросов и журналирование критически важных действий. Так как система работает с персональными и финансовыми данными, нарушение требований безопасности недопустимо и должно рассматриваться как один из основных рисков проекта [4,5].

Важным требованием является сопровождаемость. Разрабатываемая система должна быть организована таким образом, чтобы внесение изменений в один функциональный блок не требовало глубокой переработки остальных компонентов. Это особенно актуально для продукта, который в дальнейшем может расширяться новыми функциями, включая бонусную систему, корпоративные тарифы, планирование поездок и интеграцию с внешними платформами [4].

С учетом выбранного архитектурного подхода к системе можно сформулировать и инфраструктурные требования. Серверная часть должна поддерживать контейнеризованное развертывание, конфигурирование через переменные окружения и базовые средства логирования. Для межсервисного взаимодействия должны использоваться понятные контракты на основе API, а для хранения данных - решения, соответствующие профилю конкретного сервиса [3,5].

Дополнительно следует учитывать требования к удобству локального развертывания и тестирования. Для учебного проекта и MVP особенно важно, чтобы основные компоненты системы могли быть запущены в контролируемой среде без сложной ручной настройки. Это упрощает проверку работоспособности решения, ускоряет внесение изменений и делает архитектуру более прозрачной для последующего анализа в следующих главах.

Исходя из перечисленных требований, задача данной работы состоит в разработке минимально жизнеспособного продукта серверной части приложения заказа такси, реализованного на языке Java и построенного на основе микросервисной архитектуры. Разра-

батываемое решение должно обеспечивать обработку ключевых бизнес-сценариев предметной области, поддерживать взаимодействие между основными участниками сервиса и создавать основу для дальнейшего развития системы [1,5].

Для достижения поставленной цели необходимо решить следующие подзадачи:

1. Проанализировать предметную область сервисов заказа такси и выделить основные бизнес-сущности и процессы.
2. Рассмотреть особенности проектирования высоконагруженных серверных систем и определить значимые нефункциональные требования.
3. Выполнить сравнительный анализ монолитной и микросервисной архитектур и обосновать выбор микросервисного подхода.
4. Определить состав основных микросервисов серверной части и границы их ответственности.
5. Спроектировать взаимодействие сервисов, механизмы хранения данных и способы интеграции с внешними компонентами.
6. Реализовать минимально жизнеспособный продукт серверной части приложения на Java с учетом требований производительности, безопасности и дальнейшего расширения.

Таким образом, в первой главе были определены особенности предметной области, выявлены технические ограничения и сформулированы требования, которым должен соответствовать продукт серверной части приложения такси. Однако перед переходом к проектированию необходимо также рассмотреть существующие решения на рынке и обосновать целесообразность собственной разработки.

1.5 Анализ существующих решений на рынке такси-сервисов

На сегодняшний день рынок такси-сервисов представлен как крупными международными платформами, так и локальными решениями. Среди международных игроков наиболее известны Uber, Bolt и сервисы компании Lyft. В странах СНГ сильные позиции занимает Яндекс.Такси. Каждая из этих платформ имеет свои сильные и слабые стороны, анализ которых необходим для понимания места разрабатываемого продукта на рынке [4].

Uber является крупнейшим в мире агрегатором такси, работающим более чем в 70 странах. Платформа предоставляет широкий спектр услуг: от классических поездок до доставки еды. Uber разработал собственный алгоритм динамического ценообразования, доказавший свою эффективность в условиях пиковых нагрузок. Однако основным недостатком Uber с точки зрения локальных рынков является зависимость от глобальной инфраструктуры: платформа использует собственные платёжные шлюзы, картографические сервисы и системы идентификации, которые в случае введения санкций или ограничений могут перестать работать на определённой территории.

Яндекс.Такси занимает лидирующие позиции в странах СНГ и Восточной Европы. Платформа глубоко интегрирована с экосистемой Яндекса: картами, навигацией, платёжной системой. Это даёт преимущество в точности маршрутизации и удобстве оплаты для пользователей из этих стран. Однако привязка к экосистеме Яндекса также является ограничением: API Яндекс.Такси закрыт для внешних разработчиков, что делает невоз-

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		14

возможным создание собственного клиентского приложения на базе этой платформы.

Bolt - европейский агрегатор такси, активно расширяющий присутствие в Восточной Европе. Платформа делает акцент на более низких комиссиях для водителей, что позволяет удерживать конкурентоспособные цены. Как и Uber, Bolt использует глобальную инфраструктуру, что создаёт те же риски в случае введения ограничений.

Общей проблемой перечисленных платформ является их зависимость от внешнеполитической ситуации. В условиях санкционного давления, введённого в последние годы, многие западные сервисы ограничили доступ к своим API, прекратили приём платежей через отдельные платёжные системы или полностью приостановили работу в некоторых странах. Это создало потребность в локальных решениях, не зависящих от внешних ограничений. Собственная платформа позволяет независимо выбирать платёжные шлюзы, картографические сервисы, центры обработки данных и прочие компоненты инфраструктуры.

Кроме политических факторов, существует и технологическая мотивация для разработки собственного решения. Коммерческие агрегаторы не предоставляют доступа к своему исходному коду и API для создания независимых клиентов. Это делает невозможным внедрение специфических функций, которые могут потребоваться локальному заказчику: особые тарифные планы, интеграция с корпоративными системами учёта, нестандартные сценарии назначения водителей или интеграция с местными картографическими сервисами. Разработка собственной платформы снимает все эти ограничения.

Таким образом, несмотря на наличие крупных игроков на рынке такси-сервисов, разработка собственного решения остаётся актуальной задачей. Собственная платформа не подвержена рискам, связанным с санкционными ограничениями, и может быть полностью адаптирована под требования локального рынка и конкретного заказчика. Это обосновывает целесообразность выполнения данного дипломного проекта.

2. ВЫБОР ИСПОЛЬЗУЕМЫХ ТЕХНОЛОГИЙ

2.1 Обзор языков программирования и платформ для серверной разработки

Разработка серверной части приложения такси относится к классу задач, для которых особенно важны производительность, устойчивость к отказам, расширяемость и поддержка большого числа одновременных запросов. Подобные требования характерны для высоконагруженных ИС, где ошибка в проектировании приводит не только к ухудшению качества обслуживания, но и к нарушению работы ключевых бизнес-процессов [2].

Для реализации серверной части могут использоваться различные языки программирования и платформы. Наиболее распространенными вариантами являются Java, Kotlin, C#, JavaScript и Go. Каждый из них обладает собственными преимуществами, однако при проектировании корпоративных ИС важно учитывать зрелость экосистемы, наличие средств для работы с БД, поддержку тестирования, безопасность и возможности интеграции с внешними API.

Язык Java традиционно применяется для создания серверных приложений, поскольку он обладает устойчивой стандартной библиотекой, развитой экосистемой и большим количеством готовых решений для построения многослойной архитектуры. Дополнительным преимуществом является переносимость байт-кода между различными платформами и наличие зрелых средств разработки на уровне JDK и сопутствующих инструментов [4].

Среди серверных платформ Java особое место занимают решения, основанные на Spring. Данная экосистема позволяет строить приложения с четким разделением ответственности, использовать механизмы внедрения зависимостей, реализовывать REST API и упрощать работу с транзакциями, безопасностью и конфигурацией. Для учебного и прикладного проекта это снижает объем рутинного кода и делает структуру проекта более понятной [3].

В сравнении с Java, языки с более низким порогом входа, например JavaScript, удобны для быстрого прототипирования, но не всегда обеспечивают такой же уровень структурированности и зрелости в части enterprise-разработки. Go, в свою очередь, дает хорошие результаты при построении компактных высокопроизводительных сервисов, однако экосистема прикладных библиотек для типичных задач корпоративной ИС обычно менее разнообразна, чем у Java [2].

Отдельно следует отметить платформу .NET и язык C#. Данное решение также широко используется для серверной разработки и обладает развитым набором средств для работы с web-приложениями, БД и механизмами безопасности. Однако в рамках данной работы использование Java выглядит более логичным, так как выбранная предметная область лучше соотносится с существующей экосистемой Spring и большим количеством типовых архитектурных решений, описанных в специализированной литературе.

Kotlin часто рассматривается как альтернатива Java благодаря лаконичному синтаксису и совместимости с JVM. Тем не менее при реализации серверной части приложения такси выбор Java является более оправданным, поскольку для него имеется больше типовых учебных примеров, стабильных практик проектирования и накопленного опыта

использования в микросервисных системах.

При анализе платформ для серверной разработки также важно учитывать инструменты сборки, управления зависимостями и тестирования. Для Java-проектов такие задачи обычно решаются с помощью зрелых средств, которые хорошо интегрируются со Spring Boot и позволяют организовать воспроизводимую сборку, единый подход к конфигурации и автоматизацию проверок. Это особенно значимо для проекта, в котором несколько сервисов должны быть связаны общими соглашениями по API и едиными требованиями к качеству кода.

Таким образом, Java остается рациональным выбором для серверной части приложения такси. Она обеспечивает баланс между производительностью, сопровождаемостью и доступностью библиотек, а также хорошо сочетается с архитектурой, в которой отдельные функциональные блоки выделяются в независимые микросервисы [1].

2.2 Выбор и обоснование стека технологий (Java, Spring Boot)

При проектировании серверной части приложения такси необходимо не только выбрать язык программирования, но и определить набор технологий, который обеспечит быстрое создание прототипа, удобство сопровождения и возможность последующего расширения. В рамках данной работы в качестве основы выбран стек Java и Spring Boot, так как он позволяет реализовать типовую серверную ИС без избыточной сложности и с соблюдением распространенных архитектурных практик.

Spring Boot представляет собой подход к построению приложений на базе Spring, ориентированный на минимизацию конфигурации и ускорение запуска проекта. По документации платформы, Spring Boot упрощает создание автономных приложений, предоставляет готовые настройки для типичных сценариев и поддерживает построение сервисов, которые удобно разворачивать в контейнерной среде [5].

Для серверной части приложения такси это особенно важно, поскольку система должна включать обработку запросов пользователей, управление заказами, работу с авторизацией, интеграцию с внешними сервисами и обмен данными в формате JSON. Spring Boot предоставляет средства для организации контроллеров, сервисного слоя и доступа к данным, что соответствует слоистой структуре приложения и упрощает разделение ответственности между компонентами [5].

Еще одним преимуществом Spring Boot является удобство интеграции с типовыми инфраструктурными решениями. Для серверной части приложения это означает возможность подключать средства логирования, мониторинга, проверки состояния сервисов и внешние конфигурационные источники без существенного усложнения кода. В результате разработка концентрируется на предметной логике, а не на ручной сборке вспомогательных механизмов.

Существенным преимуществом Spring Boot является наличие встроенных механизмов для разработки REST API. Для приложения такси это означает возможность организовать обмен данными между клиентской и серверной частями в виде набора понятных HTTP-методов и ресурсов, что облегчает сопровождение интерфейсов и интеграцию с мобильными клиентами.

Отдельного внимания заслуживает поддержка безопасности. В ИС, где обрабатываются учетные данные пользователей и сведения о поездках, необходимо реализовать аутентификацию, авторизацию и защиту от несанкционированного доступа. Экосистема Spring предоставляет готовые средства, которые позволяют выстроить обработку JWT и контролировать доступ к защищенным ресурсам без ручной реализации базовых механизмов безопасности [5].

Практическая ценность Spring Boot проявляется и в том, что он хорошо поддерживает принцип разделения приложения на независимые слои. Контроллеры отвечают за прием и выдачу HTTP-запросов, сервисы содержат бизнес-логику, а уровень доступа к данным изолирует работу с БД. Такая структура соответствует принципам чистой архитектуры и упрощает последующее тестирование и сопровождение системы [4].

Еще одним аргументом в пользу выбранного стека является высокая распространенность Spring Boot в практике серверной разработки. Наличие большого числа документации, примеров и типовых решений снижает риск архитектурных ошибок и ускоряет выполнение проекта. Кроме того, подобный стек хорошо согласуется с микросервисным подходом, поскольку облегчает создание независимых сервисов с четко определенными API [1].

Таким образом, сочетание Java и Spring Boot обеспечивает достаточный уровень технологической зрелости для разработки серверной части приложения такси. Выбранный стек позволяет сосредоточиться на реализации предметной логики, не расходуя избыточные ресурсы на инфраструктурные детали, и при этом сохраняет возможности для дальнейшего развития системы [5].

2.3 Анализ систем управления базами данных и средств кэширования

Серверная часть приложения такси должна хранить данные о пользователях, заказах, маршрутах, статусах поездок, платежах и истории действий. Для таких задач требуется БД, которая обеспечивает целостность данных, поддержку транзакций и возможность выполнения сложных запросов. В большинстве случаев для этих целей применяются реляционные СУБД, поскольку именно они лучше всего подходят для работы с взаимосвязанными сущностями и строгими ограничениями целостности [6].

Среди реляционных СУБД особое место занимает PostgreSQL. Согласно документации, данная СУБД поддерживает транзакционность, индексацию, расширяемые типы данных и механизмы, необходимые для надежного хранения бизнес-данных [6]. Для приложения такси это важно, так как заказ, пользователь, водитель и платеж должны быть согласованы между собой и сохраняться без потери связности.

PostgreSQL также удобна тем, что позволяет применять сложные запросы и гибко организовывать схему данных. Это полезно при работе с отчетами, фильтрацией поездок, выборкой истории заказов и анализом активности пользователей. При проектировании серверной части подобная СУБД обеспечивает достаточную надежность и хорошо сочетается с ORM-подходом, который часто используется в Java-проектах.

Дополнительным преимуществом PostgreSQL является поддержка расширяемости и развитых механизмов индексации. Для сервиса такси это важно при обработке боль-

ших объемов данных, когда требуется быстро находить активные заказы, историю конкретного пользователя или ближайшие объекты по географическим признакам. Наличие таких возможностей делает PostgreSQL удобной основой для дальнейшего роста системы [6].

В качестве альтернативы для хранения данных можно рассматривать документные или key-value решения. Однако для приложения такси подобный выбор менее рационален, поскольку основная информация в системе тесно связана между собой и требует строгого контроля целостности. Именно поэтому реляционная модель данных оказывается предпочтительной для ядра серверной части.

Однако одной БД недостаточно для обеспечения высокой скорости отклика. В приложении такси часть данных используется значительно чаще, чем изменяется. К таким данным относятся справочники, настройки тарифов, сессионная информация, временные координаты и результаты часто повторяющихся проверок. Для снижения нагрузки на основную БД целесообразно применять кэширование.

Средства кэширования позволяют хранить часто запрашиваемые данные в оперативной памяти и тем самым уменьшать число обращений к основной СУБД. В качестве инструмента кэширования широко используется Redis. Документация Redis описывает поддержку различных структур данных и высокую скорость доступа, что делает данное решение практичным для сценариев, где важна малая задержка ответа [7].

Для сервиса такси Redis может быть полезен не только как обычный кэш, но и как средство хранения временных состояний, которые быстро меняются в ходе работы системы. Это относится к очередям на назначение заказа, к значениям, используемым при поиске ближайшего водителя, и к короткоживущим данным, которые нецелесообразно постоянно записывать в основную БД. Такое разделение снижает нагрузку на хранилище и повышает устойчивость сервиса к пиковым нагрузкам.

В приложении такси Redis может использоваться для хранения временных данных о состоянии заказа, ограничений на повторные запросы, промежуточных результатов вычислений и координат, обновляемых с высокой частотой. Это позволяет разгрузить PostgreSQL и повысить общую отзывчивость сервиса, особенно в периоды пикового спроса.

При сравнении БД и кэша следует учитывать, что они решают разные задачи. PostgreSQL обеспечивает долговременное и надежное хранение информации, тогда как Redis используется как быстрый слой доступа к данным. Следовательно, оптимальная схема для серверной части приложения такси предполагает совместное использование обеих технологий: основная БД отвечает за сохранность данных, а кэш - за скорость обработки наиболее частых запросов [6,7].

Таким образом, для разрабатываемой серверной части целесообразно использовать PostgreSQL в качестве основной БД и Redis в качестве средства кэширования. Такое сочетание обеспечивает баланс между надежностью хранения, производительностью и удобством последующего масштабирования серверной части приложения.

3. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СЕРВЕРНОЙ ЧАСТИ

3.1 Разработка схемы взаимодействия микросервисов

Проектирование серверной части приложения такси начинается с определения того, из каких компонентов она будет состоять и как эти компоненты будут между собой общаться. Исходя из результатов анализа, проведённого в предыдущих главах, было решено строить систему на основе микросервисной архитектуры. Логика тут простая: каждый крупный блок функциональности выносится в отдельный сервис, у каждого своя БД, и они общаются по сети. Это даёт гибкость в плане масштабирования и независимого развития отдельных частей системы [1].

В разрабатываемой системе можно выделить два типа сервисов: инфраструктурные и бизнесовые. К инфраструктурным относятся Eureka (для поиска и обнаружения сервисов), Config Server (хранилище общих настроек) и API Gateway (единая точка входа). Бизнесовых сервисов пять: сервис аутентификации, сервис пассажиров, сервис водителей, сервис поездок и сервис рейтинга. Каждый из них отвечает за свою область и имеет собственную БД (см. рисунок 3.1).

API Gateway работает на порту 8080. Он принимает все входящие запросы от клиентов, проверяет цифровые подписи и направляет запросы к нужным сервисам. Например, запросы на `/api/v1/passengers/**` идут в сервис пассажиров, на `/api/v1/drivers/**` - в сервис водителей, на `/api/v1/rides/**` - в сервис поездок. Все сервисы регистрируются в Eureka, поэтому gateway может обращаться к ним по именам, а система сама находит нужный сервис и распределяет нагрузку. Config-server, в свою очередь, хранит общие настройки для всех сервисов, и при старте каждый сервис получает конфигурацию оттуда.

Теперь про то, как сервисы общаются между собой. Тут используется два подхода: синхронный через HTTP-вызовы и асинхронный через систему обмена сообщениями. Синхронные вызовы применяются там, где нужно получить ответ прямо сейчас. Например, когда сервис аутентификации регистрирует нового пользователя, он сначала создаёт учётную запись, а потом через HTTP-запрос обращается к сервису пассажиров или сервису водителей, чтобы создать профиль. Другой пример: сервис поездок перед назначением водителя на заказ проверяет, что водитель существует и не занят. Все эти вызовы защищены: каждый сервис авторизуется как доверенное приложение, а не как конкретный пользователь.

Чтобы синхронные вызовы не ломали всю систему, если какой-то сервис временно недоступен, используются механизмы обработки ошибок: делается несколько повторных попыток, а при постоянных сбоях вызовы временно прекращаются, чтобы один упавший сервис не повалил всю систему [1].

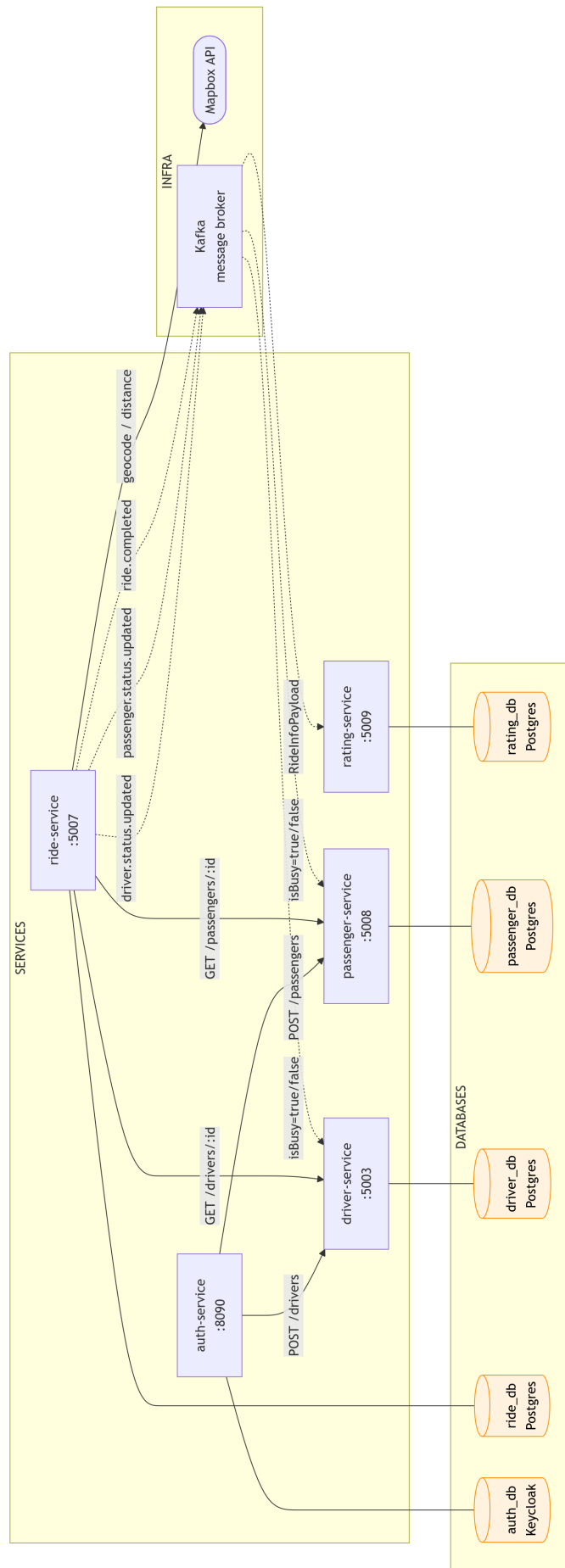


Рисунок 3.1 – Схема взаимодействия микросервисов

Асинхронное взаимодействие используется для событий, где не нужно ждать ответа прямо сейчас. Например, при создании поездки меняется статус пассажира на "занят при назначении водителя - статус водителя. Или когда поездка завершена - нужно сказать сервису рейтинга, что можно выставлять оценки. Для таких вещей сервис поездок отправляет сообщения через систему обмена сообщениями, а сервисы пассажиров, водителей и рейтинга получают эти сообщения и обрабатывают их.

Но тут есть нюанс: если отправить сообщение, а потом что-то пойдёт не так, данные могут разойтись. Чтобы такого не было, используется следующий подход: сначала данные сохраняются в БД вместе с изменением статуса поездки, а потом уже отправляются получателю. Если отправка не удалась - данные остаются в БД и будут отправлены при следующей попытке. Так гарантируется, что сообщение дойдёт хотя бы один раз [8].

Отдельно стоит сказать про авторизацию и аутентификацию. За это отвечает внешняя система управления доступом. Когда пользователь регистрируется или заходит через сервис аутентификации, тот связывается с этой системой, получает цифровой ключ доступа и передаёт его клиенту. Дальше клиент при каждом запросе прикладывает этот ключ. API Gateway проверяет ключ, достаёт оттуда идентификатор пользователя и передаёт его другим сервисам в специальном заголовке. Таким образом сервисы могут реализовать адрес /me, чтобы получить профиль текущего пользователя.

3.2 Проектирование структуры базы данных

У каждого бизнес-сервиса в системе своя БД. Такой подход снижает связанность: изменение схемы одного сервиса не затрагивает остальные [1]. Все изменения схемы отслеживаются с помощью Liquibase - инструмента миграций, который хранит список изменений (changelog) непосредственно в проекте. При запуске сервиса Liquibase проверяет, какие миграции уже применены, и выполняет только новые. Это гарантирует, что схема БД всегда соответствует версии кода, а при необходимости миграцию можно откатить.

Связи между основными сущностями системы показаны на ER-диаграмме (см. рисунок 3.2). В системе выделено четыре БД - по одной на бизнес-сервис. В сервисе водителей две таблицы: driver и car. В driver хранятся имя, фамилия, электронная почта, телефон, пол, флаги активности и занятости, а также привязка к учётной записи Keycloak. В car - цвет, марка, номер автомобиля, флаг активности и внешний ключ к таблице driver. Связь между таблицами типа "один ко многим": водитель может владеть несколькими автомобилями.

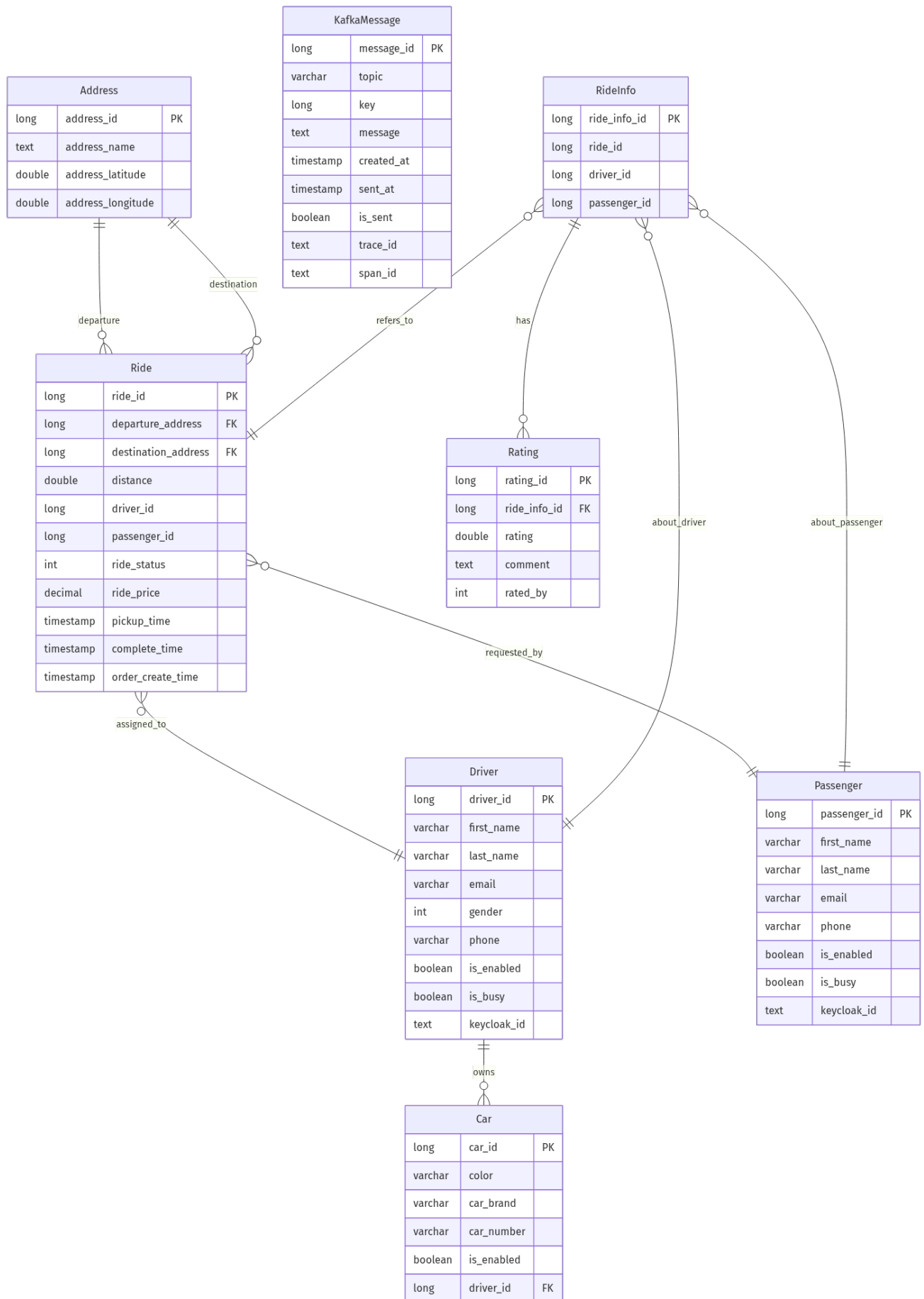


Рисунок 3.2 – ER-диаграмма базы данных

Сервис пассажиров содержит единственную таблицу `passenger` с аналогичным набором полей: имя, фамилия, `email`, телефон, флаги активности и занятости, `keycloak_id`.

Наиболее насыщенной является БД сервиса поездок, включающая таблицы `ride`, `address` и `kafka_messages`. Центральная таблица `ride` содержит: внешние ключи к таблице `address` (адреса отправления и назначения), идентификаторы водителя и пассажира в виде логических ссылок (без ограничений на уровне СУБД, поскольку эти сущности хранятся в других БД), статус поездки (12 возможных состояний, хранятся числом и преобразуются в перечисление средствами ORM), цену, дистанцию в метрах и три временные метки - создания заказа, подачи автомобиля и завершения поездки.

Таблица `address` отвечает за хранение геоданных. В ней четыре поля: название адреса (строка), широта и долгота (тип `double precision`) и суррогатный ключ. Тип `double precision` обеспечивает точность, достаточную для задач городской навигации. Широта принимает значения от -90 до 90 градусов, долгота - от -180 до 180. Координаты проверяются на уровне приложения перед записью, что исключает сохранение некорректных данных. Для ускорения поиска по названию на поле `address_name` создан индекс - это позволяет находить уже известные системе адреса без повторного обращения к картографическому сервису.

В текущей версии системы поиск ближайших водителей выполняется через внешний картографический сервис, а не через пространственные запросы к БД. Поэтому геоинформационные расширения PostgreSQL, такие как PostGIS, пока не задействованы. Однако модель данных допускает их подключение в будущем: достаточно добавить столбец геометрического типа и заполнить его на основе уже имеющихся координат без изменения существующей структуры таблиц.

При создании поездки пользователь указывает адреса отправления и назначения. Сервис поездок направляет их в картографический сервис, получает координаты и сохраняет в таблицу `address`. Затем по паре координат через тот же сервис вычисляется расстояние в метрах. Чтобы снизить число обращений к внешнему сервису, результаты всех трёх операций кэшируются в Redis: кэшируется сам адрес (название вместе с координатами), результат геокодирования (преобразование названия в координаты) и расстояние между конкретной парой адресов.

Таблица `kafka_messages` служит хранилищем исходящих сообщений. При изменении статуса поездки, требующем оповещения других сервисов (например, о занятости водителя или завершении поездки), сообщение сначала записывается в эту таблицу в рамках той же транзакции. Это гарантирует согласованность: если транзакция зафиксирована, то и данные поездки, и сообщение сохранены одновременно. Таблица содержит поля: топик, ключ (обычно идентификатор сущности), тело сообщения в JSON, время создания, время отправки и флаг `is_sent`. На полях `is_sent` и `created_at` построен составной индекс для быстрого поиска неотправленных сообщений. Раз в секунду фоновая задача выбирает строки с `is_sent = false` и публикует их в брокер сообщений, после чего помечает как отправленные. Если брокер временно недоступен, сообщения накапливаются в таблице и будут отправлены при следующем успешном цикле [8].

Поскольку сервис поездок может работать в нескольких экземплярах, фоновая задача отправки должна выполняться только на одном из них во избежание дублирования. Для этого задействован механизм распределённых блокировок `ShedLock`: перед запуском задача пытается захватить блокировку через служебную таблицу в той же БД. Если блокировка уже занята другим экземпляром, задача пропускает цикл. Раз в сутки срабатывает задача очистки, удаляющая сообщения, отправленные более суток назад, чтобы таблица

не разрасталась.

Сервис рейтинга содержит таблицы `ride_info` и `rating`. При получении сообщения о завершённой поездке создаётся запись в `ride_info` с идентификаторами поездки, водителя и пассажира. После этого каждый участник может выставить оценку: в таблицу `rating` помещается строка со ссылкой на `ride_info`, числовой оценкой (от 0 до 5), текстовым комментарием и признаком того, кто поставил оценку - пассажир или водитель. Средний рейтинг вычисляется динамически: берутся последние N оценок (по умолчанию 20) и рассчитывается среднее арифметическое.

3.3 Разработка API и протоколов обмена данными

API серверной части спроектирован в REST-стиле. Все запросы приходят на единый шлюз на порту 8080, а оттуда уже направляются к конкретным сервисам. Формат данных - JSON. Общая схема API показана в таблице 3.1.

Таблица 3.1 – Маршрутизация API Gateway

Путь	Сервис	Авторизация	Описание
/auth/**	аутентификация	нет	Регистрация и вход
/passengers/**	пассажиры	да	Работа с пассажирами
/drivers/**	водители	да	Работа с водителями
/cars/**	водители	да	Работа с машинами
/rides/**	поездки	да	Управление поездками
/ratings/**	рейтинг	да	Оценки и рейтинг

Сервис аутентификации отвечает за вход и регистрацию. У него есть открытые адреса: POST /signup (создание пользователя и его профиля), POST /signin (получение ключа доступа), POST /refresh (обновление ключа) и POST /logout (отзыв старого ключа). После регистрации сервис аутентификации через HTTP-запрос обращается к сервису пассажиров или сервису водителей, чтобы создать профиль.

Сервисы пассажиров и водителей предоставляют операции создания, чтения, обновления и удаления для своих записей. Помимо обычных адресов для работы с конкретными записями у них есть адрес /me, который возвращает профиль текущего пользователя по его идентификатору, полученному из специального заголовка. Это удобно для клиента - не нужно хранить внутренние идентификаторы. Также есть список всех записей с возможностью страничного просмотра.

Самый интересный сервис с точки зрения API - это сервис поездок. Для начала поездки клиент отправляет POST /rides с указанием адреса отправления, назначения и своего ID. Система проверяет, что пассажир не занят, преобразует адреса в координаты через картографический сервис, считает дистанцию и цену (10 условных единиц за километр), создаёт поездку и сразу переводит её в статус ожидания водителя, отправляя сообщение о том, что пассажир занят.

Дальше поездка живёт по конечному автомату, который включает 12 возможных состояний и переходы между ними. Вот как выглядит полный жизненный цикл поездки: CREATED - WAITING_FOR_DRIVER - DRIVER_ASSIGNED - ACCEPTED - ON_THE_WAY_TO_PASSENGER - PASSENGER_PICKED_UP - ON_THE_WAY -

COMPLETED - PAYMENT_PENDING - PAID - RATE. Из многих состояний можно отменить поездку (CANCELLED). (см. рисунок 3.3).

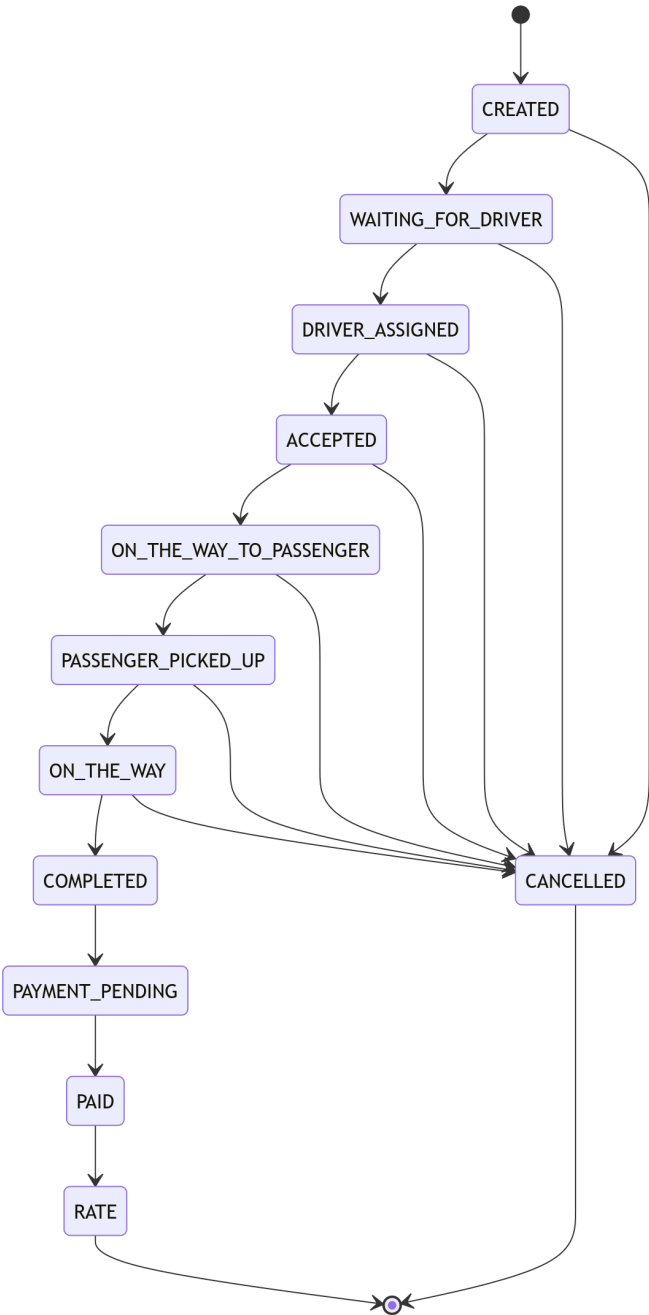


Рисунок 3.3 – Диаграмма состояний поездки

Переходы между состояниями строго контролируются: система проверяет допустимость перехода из текущего статуса в новый. Например, из CREATED разрешены только переходы в WAITING_FOR_DRIVER или CANCELLED, а из COMPLETED - только в PAYMENT_PENDING. При определённых переходах возникают побочные эффекты: при переходе в PASSENGER_PICKED_UP фиксируется время подачи, при переходе в COMPLETED - время завершения поездки, а также публикуются события освобождения водителя и пассажира в Kafka.

Сервис рейтинга работает проще всех. Его основная задача - принимать оценки после завершённых поездок. Сервис рейтинга получает сообщение о завершении поездки и создаёт запись, фиксируя этот факт. После этого пассажир или водитель могут отправить

POST /ratings с оценкой (от 0 до 5), комментарием и указанием, кто оценивает. Средний рейтинг считается программно: берутся последние N оценок (по умолчанию 20) и вычисляется среднее арифметическое. Это позволяет гибко настраивать количество учитываемых оценок.

Отдельно стоит сказать про каналы передачи сообщений, которые соединяют сервисы в единую систему. Всего их три: один меняет отметку "занят" у пассажира, второй - аналогично для водителя, третий передаёт информацию о завершённой поездке в сервис рейтинга. Каждый канал настроен так, чтобы сообщения не терялись и обрабатывались без дублирования.

					ДП.АС64.220047-05 81 00	Лист
						27
Изм.	Лист.	№ докум.	Подп.	Дата		

4. РЕАЛИЗАЦИЯ ОСНОВНЫХ МОДУЛЕЙ СИСТЕМЫ

4.1 Реализация сервиса управления заказами

Сервис поездок является центральным компонентом системы, отвечающим за весь жизненный цикл заказа - от момента создания до выставления оценки. Именно в нём сосредоточена основная бизнес-логика: создание поездки, расчёт стоимости, назначение водителя и перевод заказа между состояниями. Сервис реализован на языке Java с использованием Spring Boot 3.4 и Spring Data JPA. Данные хранятся в отдельной БД ride_db, для управления схемой которой применяется Liquibase [3,6].

Проект сервиса организован по стандартной слоистой структуре. Слои контроллеров принимает HTTP-запросы и возвращает ответы в формате JSON. Слои сервисов содержат бизнес-логику - именно в нём реализованы методы создания поездки, назначения водителя и перевода между статусами. Слои доступа к данным построены на Spring Data JPA и включают репозитории для сущностей Ride, Address и KafkaMessage. Для преобразования между сущностями и объектами передачи данных используются мапперы на базе MapStruct. Такая структура позволяет чётко разделить ответственность между компонентами и упрощает тестирование [3,4].

В основе сервиса лежат две основные сущности JPA: Ride и Address. Ride описывает поездку и связан с двумя записями Address через внешние ключи departure_address и destination_address. Address хранит геоинформацию и используется повторно: если один и тот же адрес указывается в разных заказах, новая запись в таблице address не создаётся, а используется существующая. В таблице 4.1 приведены основные поля сущности Ride.

Таблица 4.1 – Основные поля сущности Ride

Поле	Тип	Описание
ride_id	Long	Первичный ключ, автогенерация
departure_address_id	Long (FK)	Ссылка на адрес отправления
destination_address_id	Long (FK)	Ссылка на адрес назначения
distance	Double	Дистанция поездки в метрах
driver_id	Long	Идентификатор водителя (логический FK)
passenger_id	Long	Идентификатор пассажира (логический FK)
ride_status	Integer	Статус поездки, значение от 0 до 11
ride_price	BigDecimal	Стоимость в условных единицах
pickup_time	LocalDateTime	Время подачи автомобиля
complete_time	LocalDateTime	Время завершения поездки
order_create_time	LocalDateTime	Время создания заказа

Статус поездки хранится в БД в виде целого числа, а в коде преобразуется в перечисление RideStatus из двенадцати значений. Начальное состояние - CREATED. Затем заказ последовательно проходит цепочку WAITING_FOR_DRIVER, DRIVER_ASSIGNED, ACCEPTED, ON_THE_WAY_TO_PASSENGER, PASSENGER_PICKED_UP, ON_THE_WAY, COMPLETED, PAYMENT_PENDING, PAID и RATE. Из любого состояния до COMPLETED предусмотрен переход в CANCELLED. Преобразование между числом и перечислением

					ДП.AC64.220047-05 81 00	Лист
						28
Изм.	Лист	№ докум.	Подп.	Дата		

выполняет конвертер RideStatusConverter, реализующий интерфейс AttributeConverter из состава JPA. При сохранении вызывается метод convertToDatabaseColumn, записывающий порядковый номер статуса в БД. При чтении вызывается convertToEntityAttribute, восстанавливающий элемент перечисления по номеру. Такой подход позволяет хранить статус компактно - одним числом - и при этом работать с ним в коде как с типобезопасным перечислением.

Управление переходами между статусами реализовано в виде конечного автомата непосредственно в классе RideStatus. Каждый элемент перечисления хранит список разрешённых целевых состояний, в которые из него можно перейти. Метод isTransitionAllowed принимает целевой статус и проверяет его наличие в этом списке. Например, из CREATED разрешены переходы только в WAITING_FOR_DRIVER и CANCELLED; из COMPLETED - только в PAYMENT_PENDING. Метод isUpdatable определяет, можно ли изменять данные поездки в текущем статусе. Обновление адресов и цены допускается только на ранних этапах - CREATED, WAITING_FOR_DRIVER и DRIVER_ASSIGNED, - поскольку после того как водитель принял заказ, менять условия поездки уже нельзя. Диаграмма состояний с указанием всех переходов была приведена в главе 3.

API сервиса поездок предоставляет шесть основных методов, перечисленных в таблице 4.2.

Таблица 4.2 – API сервиса поездок

Метод	Путь	Назначение
GET	/api/v1/rides/{id}	Получение поездки по идентификатору
GET	/api/v1/rides	Список всех поездок с пагинацией
GET	/api/v1/rides/driver/{driverId}	Поездки конкретного водителя
GET	/api/v1/rides/passenger/{passengerId}	Поездки конкретного пассажира
POST	/api/v1/rides	Создание новой поездки
PATCH	/api/v1/rides/{id}/assign-driver	Назначение водителя
PUT	/api/v1/rides/{id}	Изменение данных поездки
PATCH	/api/v1/rides/{id}	Изменение статуса поездки

Создание поездки - наиболее сложная операция сервиса. Она реализована в методе createRide сервисного слоя и выполняется в несколько строго определённых шагов. Первым делом через Feign-клиент PassengerFeignClient проверяется, что пассажир с заданным идентификатором существует и не занят в другой активной поездке. Если пассажир занят, выбрасывается исключение, и заказ не создаётся - это предотвращает ситуацию, когда один человек создаёт несколько параллельных заказов. Затем проверяется, что адреса отправления и назначения не совпадают, иначе поездка теряет смысл.

После успешной валидации начинается работа с геоданными. Каждый адрес, переданный клиентом в текстовом виде, направляется в картографический сервис. Сервис возвращает координаты, которые сохраняются в таблицу address. Если такой адрес уже существует в БД, новая запись не создаётся - используется существующая. По полученным координатам отправления и назначения через тот же картографический сервис вычисляется расстояние в метрах. Итоговая стоимость рассчитывается по формуле (4.1):

$$P = D \cdot T / 1000 \quad (4.1)$$

где P – стоимость поездки в условных единицах;

D – дистанция поездки в метрах;

T – тариф, равный 10 условных единиц за километр.

После расчёта цены создаётся объект Ride. В него записываются ссылки на адреса, дистанция, идентификатор пассажира, рассчитанная цена и текущее время в поле `order_create_time`. Поле `ride_status` устанавливается в `CREATED`. Объект сохраняется в БД через репозиторий. Сразу после сохранения статус переводится в `WAITING_FOR_DRIVER` - заказ готов к тому, чтобы на него назначили водителя. В рамках той же транзакции в таблицу `kafka_messages` добавляется запись с топиком `passenger-busy-topic` и значением `true`. Это событие позже будет доставлено в сервис пассажиров, и пассажир будет отмечен как занятый.

Назначение водителя выполняется отдельным методом `assignDriver`. На вход подаются идентификатор поездки и идентификатор водителя. Метод извлекает поездку из БД и проверяет, что она находится в статусе `WAITING_FOR_DRIVER` - назначать водителя на поездку, которая уже выполняется или завершена, бессмысленно и запрещено. Затем через `DriverFeignClient` запрашивается информация о водителе. Проверяются три условия: водитель должен быть активен (поле `is_enabled` = `true`), не занят в другой поездке (`is_busy` = `false`) и иметь хотя бы один зарегистрированный автомобиль. Если хотя бы одно условие не выполнено, выбрасывается исключение. При успешной проверке в запись Ride записывается `driver_id`, статус меняется на `DRIVER_ASSIGNED`, а в `kafka_messages` добавляется событие `driver-busy-topic` со значением `true`.

Изменение статуса поездки выполняется методом `updateRideStatus`. Он принимает идентификатор поездки и целевой статус. Поездка извлекается из БД, после чего вызывается `isTransitionAllowed` для проверки допустимости перехода. Если переход разрешён, статус обновляется. При определённых переходах возникают побочные эффекты, реализованные в виде отдельных действий в том же методе.

При переходе в `PASSENGER_PICKED_UP` в поле `pickup_time` записывается текущее время - это отметка о моменте, когда водитель забрал пассажира. При переходе в `COMPLETED` заполняется `complete_time`, а в `kafka_messages` добавляются два события с топиками `driver-busy-topic` и `passenger-busy-topic` со значением `false`: и водитель, и пассажир освобождаются и могут участвовать в новых поездках. При переходе в `RATE` в `kafka_messages` добавляется событие `ride-completed-topic`, содержащее идентификаторы поездки, водителя и пассажира, - оно будет доставлено в сервис рейтинга для создания записи `ride_info`. При переходе в `CANCELLED` также публикуются события освобождения участников, если они были назначены к моменту отмены. Без этого шага отменённая поездка заблокировала бы водителя и пассажира навсегда.

Для взаимодействия с другими сервисами через HTTP используются Feign-клиенты из состава Spring Cloud OpenFeign. Сервис поездок содержит два таких клиента: `DriverFeignClient` и `PassengerFeignClient`. Каждый из них описан как интерфейс с аннотацией `@FeignClient`, в которой указано имя сервиса, зарегистрированное в Eureka, - `driver-service` и `passenger-service` соответственно. Методы интерфейса аннотированы стандартными аннотациями Spring MVC: `@GetMapping`, `@PathVariable` и другими. При вызове метода Feign автоматически строит HTTP-запрос к нужному сервису, сериализует параметры и десериализует ответ. Благодаря интеграции с Eureka не требуется указывать конкретный адрес - Feign получает список доступных экземпляров сервиса от Eureka и распределяет запросы между ними.

Все Feign-клиенты сконфигурированы с использованием библиотеки `Resilience4J`.

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		30

Для каждого клиента заданы две политики. Первая - повторные попытки: если вызов завершился ошибкой, предпринимается ещё несколько попыток с возрастающей задержкой между ними. Вторая - circuit breaker: если доля ошибочных вызовов превышает заданный порог, цепь размыкается, и последующие вызовы немедленно возвращают ошибку без фактического обращения к целевому сервису. Через некоторое время цепь переходит в полукоткрытое состояние, и несколько пробных запросов проверяют, восстановилась ли работоспособность. Такая схема предотвращает каскадные отказы: временная недоступность одного сервиса не приводит к исчерпанию ресурсов у вызывающего [1,3].

На уровне контроллера все входящие данные проходят валидацию. Для этого используются аннотации из пакета `jakarta.validation`: `@NotNull`, `@NotBlank`, `@Positive` и другие. Если данные не проходят проверку, Spring автоматически возвращает клиенту ответ с кодом 400 и описанием ошибок. Для обработки исключений бизнес-логики в сервисе реализован глобальный обработчик с аннотацией `@ControllerAdvice`. Каждое исключение преобразуется в стандартизованный JSON-ответ с кодом ошибки, текстовым описанием и временной меткой. Такой подход делает API предсказуемым для клиентов и упрощает отладку.

Помимо сервиса поездок, в системе реализованы сервисы водителей и пассажиров. Они устроены значительно проще, поскольку выполняют в основном CRUD-операции над своими сущностями. Сервис водителей работает с двумя сущностями: `Driver` и `Car`, связанными отношением "один ко многим". Сервис пассажиров оперирует единственной сущностью `Passenger`. Оба сервиса предоставляют стандартные методы создания, чтения, обновления и удаления записей, а также метод `/me`, возвращающий профиль текущего пользователя по идентификатору из заголовка `X-User-Id`. Удаление реализовано как мягкое: запись не удаляется физически, а помечается как неактивная установкой флага `is_enabled` в `false`. Это позволяет сохранить историю поездок и избежать нарушения ссылочной целостности между сервисами [4].

Сервис рейтинга отвечает за хранение и обработку оценок. Он содержит сущности `RideInfo` и `Rating`, связанные отношением "один ко многим". `RideInfo` создаётся при получении события о завершении поездки из топика `ride-completed-topic`. После этого водитель и пассажир могут через POST-запрос добавить оценку - числовое значение от 0 до 5 и текстовый комментарий. API сервиса также предоставляет методы для получения среднего рейтинга водителя или пассажира. Средний рейтинг вычисляется динамически: из БД выбираются последние 20 оценок для заданного участника и рассчитывается среднее арифметическое.

4.2 Разработка модуля обработки геолокации и поиска ближайших водителей

Работа с геоданными - одна из ключевых особенностей сервиса такси. Пользователь указывает адреса в привычном текстовом виде, однако системе для расчёта стоимости, построения маршрута и передачи информации водителю необходимы географические координаты. Эту задачу решает модуль геолокации, взаимодействующий с картографическим сервисом и обеспечивающий хранение и кэширование полученных данных [6].

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		31

Взаимодействие с сервисом Mapbox инкапсулировано в отдельном классе MapboxClient. Он содержит два публичных метода. Первый метод - geocode - принимает название адреса строкой и возвращает координаты: широту (latitude) и долготу (longitude) в виде чисел типа Double. Второй метод - getDistance - принимает две пары координат и возвращает расстояние между ними в метрах, также в виде Double. HTTP-запросы к Mapbox выполняются с помощью RestClient из состава Spring Boot [5].

Для работы с Mapbox требуется API-ключ, который передаётся в заголовке каждого запроса. Ключ хранится в конфигурации сервиса и не попадает в исходный код. В конфигурационном файле заданы базовые URL API геокодирования и API маршрутизации, а также параметры по умолчанию, такие как ограничение области поиска [6].

При вызове geocode формируется GET-запрос к API геокодирования Mapbox. В URL передаётся параметр поиска, содержащий название адреса. Сервис возвращает массив совпадений в формате JSON, из которого клиентский код извлекает координаты первого - наиболее релевантного - результата. Если сервис не находит ни одного совпадения, генерируется исключение, и создание заказа прерывается.

Метод getDistance формирует запрос к API маршрутизации Mapbox. В запросе передаются две точки с координатами, а также параметр, указывающий, что требуется автомобильный маршрут. Сервис возвращает JSON-ответ, содержащий секцию с рассчитанной дистанцией в метрах - именно это значение извлекается и возвращается клиентским кодом.

Следует пояснить, почему для расчёта расстояния используется внешний сервис, а не простая математическая формула. Расстояние между двумя точками на поверхности Земли можно вычислить по формуле гаверсинусов (Haversine), которая определяет кратчайшее расстояние по дуге большого круга:

$$d = 2 \cdot R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right) \quad (4.2)$$

где R – средний радиус Земли, равный 6371000 м;

ϕ_1, ϕ_2 – широты первой и второй точек в радианах;

λ_1, λ_2 – долготы первой и второй точек в радианах;

$\Delta\phi = \phi_2 - \phi_1$;

$\Delta\lambda = \lambda_2 - \lambda_1$.

Формула (4.2) даёт расстояние по прямой. Однако в приложении такси пассажира интересует не это расстояние, а фактический путь по дорогам. Именно дорожная дистанция определяет длительность поездки и её стоимость. Mapbox Directions API учитывает дорожную сеть, разрешённые направления движения, скоростные ограничения и даже рельеф местности, возвращая реальную дистанцию по автомобильным дорогам. Разница между гаверсинусами и дорожной дистанцией может составлять десятки процентов, особенно в городах с плотной застройкой и сложной дорожной сетью. Поэтому в проекте используется именно Mapbox API [6].

Запросы к картографическому сервису выполняются по сети и имеют заметную задержку - от десятков до сотен миллисекунд. При этом результаты для одних и тех же адресов не меняются между вызовами: координаты здания на карте постоянны, а расстояние по дорогам между двумя точками меняется редко. Поэтому в системе реализовано трёхуровневое кэширование с использованием Redis [7].

Первый уровень - кэширование самого адреса как сущности. Когда сервис получает название адреса, он сначала ищет его в таблице address по полю address_name. Если

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		32

запись найдена, она возвращается немедленно, без запроса к картографическому сервису. Для быстрого поиска по названию на поле `address_name` создан индекс. Второй уровень - кэширование результата геокодирования. Даже если адрес ещё не сохранён в БД, координаты, полученные для конкретного названия, помещаются в Redis с ограниченным временем жизни. При повторном запросе того же названия координаты достаются из кэша. Третий уровень - кэширование дистанции. Расстояние между конкретными координатами сохраняется в Redis, и при повторном расчёте для той же пары точек сетевой запрос не выполняется.

Технически кэширование реализовано с помощью механизма аннотаций Spring Cache. В конфигурационном классе сервиса поездок определён менеджер кэша, использующий Redis в качестве хранилища. Для каждого уровня кэширования заведён свой кэш с уникальным именем: `addresses`, `geocode` и `distance`. Методы сервисного слоя, отвечающие за получение адресов, координат и дистанций, помечены аннотацией `@Cacheable` с указанием имени соответствующего кэша. Ключ кэша формируется автоматически на основе значений параметров метода. При вызове такого метода Spring сначала проверяет наличие значения в Redis. Если значение найдено - оно возвращается, минуя тело метода. Если нет - метод выполняется, а его результат помещается в Redis. Время жизни записей задано разным для разных кэшей: адреса хранятся дольше, результаты геокодирования - меньше, а дистанции имеют самое короткое время жизни, поскольку дорожная ситуация может измениться [7].

Координаты, полученные от картографического сервиса и сохранённые в таблице `address`, имеют тип `double precision`. Широта принимает значения в диапазоне от -90 до 90 градусов, долгота - от -180 до 180. Перед сохранением оба значения проверяются на принадлежность этим диапазонам. Проверка выполняется на уровне приложения, в сервисном слое, до передачи данных в репозиторий. Если координаты выходят за допустимые границы, операция прерывается с исключением. Это исключает сохранение заведомо некорректных данных, которые могли бы возникнуть, например, при ошибке картографического сервиса.

В текущей версии системы поиск ближайших водителей также реализован через картографический сервис. При запросе на назначение водителя сервис поездок получает список свободных водителей и передаёт их координаты вместе с координатами пассажира в картографический сервис, который упорядочивает водителей по расстоянию. Такой подход не требует пространственных запросов к БД и использования расширений вроде PostGIS. Однако модель данных спроектирована так, что в будущем можно добавить к таблице `address` столбец геометрического типа и заполнить его значениями из уже имеющихся полей широты и долготы. Это потребует минимальных изменений в коде и схеме БД [6].

4.3 Система авторизации и аутентификации на основе JWT

Вопросы безопасности в сервисе такси стоят особенно остро, поскольку система работает с персональными данными пассажиров и водителей, историей поездок и платёжной информацией. Для обеспечения защиты в проекте используется Keycloak - сер-

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		33

вер авторизации с открытым исходным кодом, работающий по протоколу OAuth2 и использующий JWT в качестве формата токенов доступа. Keycloak выполняет роль единого Identity Provider: только он хранит учётные данные пользователей и управляет ролями, а все остальные сервисы лишь проверяют подлинность предъявленных токенов [5].

В Keycloak для приложения создан отдельный realm, изолирующий его настройки от других систем, которые могут работать на том же сервере. В рамках realm определены клиенты для каждого микросервиса, включая API Gateway. Также определены роли, отражающие уровни доступа в системе. Роль USER назначается каждому зарегистрированному пользователю автоматически и предоставляет базовые права на использование системы. Роли DRIVER и PASSENGER указывают на бизнес-роль пользователя и определяют, с каким сервисом - водителей или пассажиров - он работает. Роль ADMIN предназначена для административных операций. Отдельная роль SERVICE используется для межсервисной авторизации: когда один микросервис обращается к другому, он действует от имени этой роли, а не от имени конечного пользователя.

Сервис аутентификации предоставляет клиентам четыре основных метода, перечисленных в таблице 4.3.

Таблица 4.3 – API сервиса аутентификации

Метод	Путь	Назначение
POST	/api/v1/auth/signup	Регистрация пользователя
POST	/api/v1/auth/signin	Вход и получение токенов
POST	/api/v1/auth/refresh	Обновление access-токена
POST	/api/v1/auth/logout	Отзыв refresh-токена

Процесс регистрации нового пользователя включает взаимодействие трёх компонентов: сервиса аутентификации, Keycloak и одного из бизнес-сервисов. Клиент отправляет POST-запрос на /api/v1/auth/signup, передавая в теле имя, фамилию, адрес электронной почты, пароль, номер телефона и желаемую бизнес-роль - DRIVER или PASSENGER. Сервис аутентификации принимает запрос, валидирует входные данные и приступает к созданию учётной записи.

На первом шаге через Keycloak Admin Client создаётся пользователь в Keycloak. Для этого используется специальная учётная запись администратора realm, настроенная в конфигурации сервиса. Keycloak Admin Client принимает данные пользователя и создаёт запись во внутреннем хранилище Keycloak, возвращая уникальный идентификатор нового пользователя. На втором шаге пользователю назначаются роли: обязательно роль USER, а также выбранная бизнес-роль - DRIVER или PASSENGER. Назначение ролей также выполняется через Keycloak Admin Client. На третьем шаге сервис аутентификации через HTTP-запрос обращается к сервису водителей или пассажиров - в зависимости от выбранной роли - и создаёт там профиль пользователя, передавая его идентификатор из Keycloak, имя, фамилию, email и телефон. Все три шага образуют единый бизнес-процесс регистрации, и если на любом из них происходит ошибка, процесс прерывается.

Для входа в систему клиент отправляет POST /signin с email и паролем. Сервис аутентификации направляет эти учётные данные в Keycloak, используя стандартный механизм OAuth2 - Password Grant Flow. В запросе к Keycloak передаются client_id, client_secret, имя пользователя, пароль и score. Keycloak проверяет учётные данные и, если они корректны, формирует и возвращает два JWT-токена: access-токен и refresh-токен. Access-токен имеет короткое время жизни и содержит в себе идентификатор пользователя в поле

sub, его роли в поле realm_access, а также имя клиента, для которого выдан токен. Refresh-токен имеет более длительное время жизни и используется только для получения нового access-токена, когда срок действия текущего истёк. Оба токена возвращаются клиенту в JSON-ответе.

Для обновления access-токена клиент отправляет POST /refresh, передавая refresh-токен. Сервис аутентификации перенаправляет запрос в Keycloak, который проверяет refresh-токен и, если он действителен, выпускает новую пару токенов. Для завершения сессии клиент отправляет POST /logout с refresh-токеном, и Keycloak отзывает его, делая невозможным дальнейшее обновление access-токена.

Проверка токенов при каждом запросе выполняется не сервисом аутентификации, а непосредственно на уровне API Gateway. Шлюз сконфигурирован как OAuth2 Resource Server. В его настройках указан адрес Keycloak и публичный ключ для проверки подписи JWT. При получении запроса шлюз извлекает токен из заголовка Authorization, проверяет его подпись, срок действия и соответствие эмитента. Если проверка не пройдена, шлюз возвращает ответ с кодом 401, и запрос не доходит до бизнес-сервисов. Все пути, начинающиеся с /api/v1/auth и /actuator, исключены из проверки - они должны быть доступны без авторизации [5].

После успешной проверки шлюз выполняет дополнительный шаг: он извлекает из токена идентификатор пользователя из поля sub и помещает его в HTTP-заголовок X-User-Id. Этот заголовок добавляется к запросу при его перенаправлении к целевому сервису. Реализовано это в фильтре AddUserIdHeaderFilter, который является компонентом Spring Cloud Gateway. Фильтр выполняется после проверки токена и перед отправкой запроса нижестоящему сервису. Благодаря этому заголовку любой бизнес-сервис может определить, от имени какого пользователя пришёл запрос.

Наличие заголовка X-User-Id в каждом запросе позволило реализовать в сервисах водителей и пассажиров удобный адрес /me. Клиенту не нужно знать внутренний идентификатор пользователя в системе - достаточно обратиться к /me, и сервис, прочитав X-User-Id из заголовка, найдёт соответствующий профиль и вернёт его. Это упрощает клиентский код и повышает безопасность, поскольку идентификаторы не передаются в URL [4].

Межсервисное взаимодействие защищено отдельным механизмом. Когда один сервис обращается к другому через Feign-клиент, он не использует токен конечного пользователя. Вместо этого применяется Client Credentials Flow - ещё один стандартный механизм OAuth2, предназначенный для взаимодействия серверов без участия пользователя. Сервис отправляет в Keycloak свои client_id и client_secret и получает служебный access-токен, ассоциированный с ролью SERVICE. Этот токен передаётся в заголовке Authorization Feign-запроса. Целевой сервис проверяет токен так же, как и пользовательский, но видит в нём роль SERVICE и предоставляет доступ в соответствии с этой ролью. Таким образом достигается баланс между удобством межсервисного взаимодействия и контролем доступа.

4.4 Интеграция брокера сообщений для обработки событий в реальном времени

В системе заказа такси многие действия порождают события, о которых должны быть уведомлены другие сервисы. При создании поездки пассажир переходит в состояние

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		35

занятости. При назначении водителя он также помечается занятым. При завершении или отмене поездки оба участника освобождаются, а сервис рейтинга получает информацию для создания записи о выполненном заказе. Для передачи таких событий в проекте используется брокер сообщений Apache Kafka - распределённая платформа потоковой обработки, обеспечивающая надёжную асинхронную доставку сообщений между компонентами системы [8].

Kafka была выбрана по нескольким причинам. Во-первых, она обеспечивает сохранение сообщений на диске и их репликацию между брокерами, что гарантирует доставку даже при временных сбоях. Во-вторых, Kafka поддерживает модель издатель-подписчик, при которой одно сообщение может быть доставлено нескольким независимым получателям. В-третьих, Kafka хранит сообщения упорядоченно в рамках одного раздела топика, что позволяет восстанавливать последовательность событий при необходимости.

В системе определены три топика, сведённые в таблицу 4.4.

Таблица 4.4 – Топики Kafka

Топик	Отправитель	Получатель	Назначение
passenger-busy-topic	сервис поездок	сервис пассажиров	Изменение статуса занятости
driver-busy-topic	сервис поездок	сервис водителей	Изменение статуса занятости
ride-completed-topic	сервис поездок	сервис рейтинга	Уведомление о завершении поездки

Топики passenger-busy-topic и driver-busy-topic устроены идентично. Ключом сообщения выступает идентификатор пассажира или водителя (Long), а телом - булево значение, инкапсулированное в JSON. Если значение равно true, участник помечается как занятый; если false - как свободный. Использование идентификатора в качестве ключа гарантирует, что все сообщения, относящиеся к одному участнику, попадают в один и тот же раздел топика и обрабатываются в порядке поступления - это важно, чтобы статусы занятости не перепутались.

Топик ride-completed-topic использует идентификатор поездки в качестве ключа. Телом сообщения служит JSON-объект с тремя полями: rideId, driverId и passengerId. Этих данных достаточно сервису рейтинга, чтобы создать запись ride_info, к которой впоследствии будут привязаны оценки.

Отправка сообщений реализована с применением паттерна Outbox. Его идея состоит в том, что сообщение не отправляется в Kafka непосредственно из сервисного слоя после изменения данных. Вместо этого оно сначала записывается в таблицу kafka_messages - в той же транзакции БД, что и изменение данных поездки. Транзакция либо фиксируется целиком - и тогда и новые данные, и сообщение гарантированно сохранены, - либо откатывается, и тогда не сохраняется ни то, ни другое. Промежуточных состояний не возникает. Такой подход решает классическую проблему распределённых систем: когда данные уже изменены, а событие ещё не отправлено (или наоборот), что приводит к несогласованности между сервисами. С Outbox такого не происходит [8].

Таблица kafka_messages спроектирована для хранения всей необходимой информации об исходящем сообщении. Поле message_id - автоинкрементный первичный ключ. Поле topic содержит имя топика Kafka. Поле key - ключ сообщения типа Long. Поле message хранит тело сообщения в виде строки в формате JSON, для этого использован тип TEXT.

Поля `created_at` и `sent_at` - временные метки, заполняемые при создании и отправке соответственно. Поле `is_sent` - булев флаг, по умолчанию равный `false` и меняющийся на `true` после успешной отправки. Также в таблице присутствуют поля `trace_id` и `span_id`, используемые для сквозной распределённой трассировки запросов. На полях `is_sent` и `created_at` создан составной индекс: он делает эффективным основной запрос планировщика, выбирающий неотправленные сообщения в порядке их создания.

Фактическая отправка сообщений из таблицы в Kafka выполняется фоновым планировщиком `KafkaMessageScheduler`. Он реализован как метод с аннотацией `@Scheduled`, настроенной на выполнение с фиксированной задержкой. В каждом цикле планировщик выполняет запрос к репозиторию `KafkaMessage`, выбирая все записи, у которых `is_sent = false`, в порядке возрастания `created_at`. Для каждой записи вызывается метод `KafkaTemplate.send` передающий топик, ключ и тело сообщения. После успешного вызова поле `is_sent` устанавливается в `true`, а в `sent_at` записывается текущее время. Если отправка конкретного сообщения завершается ошибкой, оно остаётся в таблице с `is_sent = false` и будет повторно обработано в следующем цикле.

Так как сервис поездок может быть запущен в нескольких экземплярах - например, для повышения отказоустойчивости, - планировщик должен работать только на одном из них. Иначе два экземпляра одновременно прочитают одни и те же неотправленные сообщения и опубликуют их дважды. Проблема решается с помощью библиотеки `ShedLock`, реализующей механизм распределённых блокировок на основе БД. В конфигурации `ShedLock` задаётся поставщик блокировок, указывающий на основную БД сервиса, и создаётся служебная таблица `shedlock`. Перед выполнением метода планировщика `ShedLock` пытается вставить в эту таблицу запись с уникальным именем задачи. Если запись успешно создана - блокировка получена, и планировщик выполняет свою работу. Если запись уже существует - значит, блокировку держит другой экземпляр, и текущий цикл пропускается. По завершении метода блокировка освобождается, либо истекает по таймауту, если экземпляр аварийно завершился [8].

На стороне получателей сообщения обрабатываются с помощью классов, аннотированных `@KafkaListener` из библиотеки `Spring for Apache Kafka`. В сервисе пассажиров слушатель подписан на топик `passenger-busy-topic`. При получении сообщения он извлекает ключ - идентификатор пассажира - и значение занятости из тела, десериализуя JSON в `Boolean`. Затем через репозиторий находится запись пассажира, и поле `is_busy` обновляется в соответствии с полученным значением. Аналогично работает слушатель в сервисе водителей для топика `driver-busy-topic`. Оба слушателя спроектированы как идемпотентные: повторная обработка одного и того же сообщения не приводит к некорректному состоянию, поскольку каждая обработка просто устанавливает поле `is_busy` в переданное значение.

Слушатель в сервисе рейтинга подписан на топик `ride-completed-topic`. При получении сообщения он десериализует JSON в объект класса `RideInfoPayload`, содержащий `rideId`, `driverId` и `passengerId`. Затем через репозиторий создаётся новая запись `ride_info` с этими идентификаторами. После создания записи и водитель, и пассажир могут выставить оценки через `POST /api/v1/ratings`, указав идентификатор поездки, числовую оценку, текстовый комментарий и признак того, кто оценивает - пассажир или водитель. Слушатель также является идемпотентным благодаря уникальному ограничению на `ride_id` в таблице `ride_info`, предотвращающему создание дубликатов [8].

Для предотвращения бесконтрольного роста таблицы `kafka_messages` реализован отдельный планировщик очистки. Раз в сутки - в ночное время, когда нагрузка на систе-

му минимальна - он выполняет запрос на удаление всех записей, у которых `is_sent = true` и `sent_at` старше одного дня. Хранение отправленных сообщений дольше этого срока не требуется, поскольку все заинтересованные сервисы уже получили и обработали их.

Конфигурация Kafka задаётся в файлах настроек Spring Boot каждого сервиса, использующего брокер. В настройках указываются адреса брокеров, идентификатор группы потребителей, стратегия подтверждения доставки и параметры сериализации. Ключи сообщений сериализуются стандартным `LongSerializer`, значения - `StringSerializer`. На стороне потребителей применяются соответствующие десериализаторы. Для повышения надёжности все топики сконфигурированы с фактором репликации, обеспечивающим сохранность данных при выходе из строя одного из узлов кластера. Также настроено автоматическое создание топиков при запуске приложения, что упрощает развёртывание системы в новой среде [8].

Особенностью реализации является интеграция с системой распределённой трассировки. При сохранении сообщения в таблицу `kafka_messages` в поля `trace_id` и `span_id` записываются соответствующие идентификаторы из текущего контекста трассировки. При отправке сообщения в Kafka эти идентификаторы восстанавливаются и передаются в заголовках сообщения. На стороне потребителя они снова извлекаются и помещаются в контекст трассировки. Благодаря этому в системах мониторинга можно проследить полный путь запроса - от HTTP-вызова до обработки Kafka-сообщения и обратно.

Таким образом, выбранный подход к организации асинхронного взаимодействия обеспечивает надёжную доставку событий между сервисами без жёстких синхронных зависимостей. Паттерн Outbox гарантирует согласованность данных при любом исходе транзакции, ShedLock предотвращает дублирование сообщений при многократном запуске отправителя, а идемпотентные слушатели делают обработку устойчивой к повторным доставкам.

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		38

5. ТЕСТИРОВАНИЕ И РАЗВЕРТЫВАНИЕ СИСТЕМЫ

5.1 Модульное и интеграционное тестирование компонентов

Тестирование является неотъемлемой частью разработки любой программной системы, особенно микросервисной, где ошибка в одном компоненте может затронуть работу всей платформы. В проекте реализовано несколько уровней тестирования, охватывающих отдельные классы, группы компонентов и сквозные пользовательские сценарии [3].

Для модульного тестирования используется связка JUnit 5 и Mockito. Модульные тесты проверяют корректность работы отдельных классов в изоляции от внешних зависимостей. Например, в сервисе поездок тестируется метод `createRide` сервисного слоя: репозитории и Feign-клиенты подменяются mock-объектами, после вызова метода проверяется, что на репозитории был вызван метод `save` с корректными параметрами, а в Feign-клиент ушли правильные запросы. Mockito позволяет задать ожидаемое поведение mock-объектов и проверить факт их вызова, при этом реальные HTTP-запросы и операции с БД не выполняются. Модульные тесты выполняются быстро и не требуют запущенной инфраструктуры, поэтому их удобно прогонять при каждой сборке проекта [3].

Интеграционные тесты проверяют взаимодействие компонентов с реальной инфраструктурой. Для их выполнения используется библиотека TestContainers, которая позволяет программно запускать Docker-контейнеры с PostgreSQL и Kafka непосредственно из тестового кода. Тест, помеченный аннотацией `@Testcontainers`, при запуске создаёт временный контейнер PostgreSQL с чистой БД, накатывает на неё Liquibase-миграции, и далее тест работает с БД так же, как реальное приложение. После завершения теста контейнер уничтожается вместе с данными. Аналогично для Kafka поднимается временный контейнер, в котором создаются топики с тестовыми именами, и тесты проверяют корректность отправки и получения сообщений [3,8].

В сервисе поездок интеграционные тесты проверяют полный цикл работы с заказом. Сначала тест создаёт поездку через контроллер: передаётся запрос с адресами и идентификатором пассажира. Контроллер обращается к сервисному слою, тот обращается к Marbox-клиенту (который для тестов также подменяется mock-объектом, возвращающим заранее заданные координаты и дистанцию), после чего создаётся запись `Ride` в БД. Тест проверяет, что поездка сохранена с правильными полями, статус равен `WAITING_FOR_DRIVER`, а в таблице `kafka_messages` появилась запись с топиком `passenger-busy-topic`. Затем через контроллер выполняется назначение водителя, и тест проверяет смену статуса и появление записи для `driver-busy-topic`. После этого статус меняется несколько раз, и тест на каждом шаге проверяет корректность перехода с помощью методов конечного автомата.

Для тестирования пользовательских сценариев в проекте используется подход BDD (Behavior-Driven Development) с фреймворком Cucumber. Сценарии описываются на естественном языке в feature-файлах с помощью ключевых слов `Given`, `When`, `Then`. Например, сценарий "Создание поездки пассажиром" выглядит так: `Given` существует активный пассажир; `When` пассажир создаёт поездку с адресами "ул. Московская, 267" и "ул. Советская, 50"; `Then` поездка создана, статус равен `WAITING_FOR_DRIVER`, и пассажир поме-

чен как занятый. За каждым шагом закреплён Java-метод, реализующий соответствующее действие или проверку. Cucumber-тесты запускаются вместе с интеграционными и проверяют те же пользовательские сценарии, которые описаны в функциональных требованиях [3].

Для тестирования API-эндпоинтов применяется библиотека REST Assured. Она предоставляет удобный DSL для отправки HTTP-запросов и проверки ответов. В тесте формируется запрос с телом в формате JSON, отправляется на заданный URL, затем проверяется код ответа (например, 201 при создании или 200 при успешном запросе), наличие определённых полей в ответе и их значения. REST Assured позволяет тестировать эндпоинты на уровне HTTP, не поднимая весь HTTP-сервер, что ускоряет выполнение тестов. В связке с TestContainers это даёт возможность протестировать полный путь от HTTP-запроса до записи в БД.

Для контроля качества кода в проекте настроен Checkstyle. Конфигурация задана в файле checkstyle.xml в корне проекта и основана на Google Java Style с адаптацией под принятые в проекте соглашения. Checkstyle проверяет форматирование кода, наличие неиспользуемых импортов, длину строк и методов, соответствие Javadoc-комментариев. Проверка запускается при сборке Maven и не позволяет закоммитить код, не соответствующий стандартам. Это поддерживает единый стиль кода во всех сервисах, даже если над ними работает несколько человек [4].

Все тесты запускаются командой `mvn verify` в корневом `pom.xml` проекта. Maven последовательно проходит по всем модулям и в каждом выполняет компиляцию, модульные тесты, интеграционные тесты и проверку Checkstyle. Если любой тест не проходит или Checkstyle находит нарушения, сборка считается неудачной. Такой подход гарантирует, что любое изменение кода проходит полную проверку перед тем, как попасть в основную ветку проекта.

5.2 Контейнеризация приложения с использованием Docker

Развёртывание микросервисной системы, состоящей из восьми сервисов, пяти БД и нескольких вспомогательных служб, вручную было бы трудоёмким и чреватым ошибками. Для автоматизации этого процесса все компоненты системы упакованы в Docker-контейнеры. Docker позволяет запустить весь проект одной командой, гарантируя, что все сервисы работают в одинаковом окружении независимо от операционной системы хоста [1].

Сборка Docker-образа для каждого сервиса описана в `Dockerfile`, расположенном в директории `docker`. Файл использует многоступенчатую сборку. На первом этапе (`build stage`) используется образ Maven с JDK 17. Исходный код сервиса копируется в контейнер, и выполняется команда `mvn package -DskipTests`, собирающая исполняемый JAR-файл. На втором этапе создаётся минимальный образ на базе JRE. В него копируется только собранный JAR-файл, после чего указывается команда запуска `java -jar`. Благодаря разделению на этапы итоговый образ содержит только среду выполнения и скомпилированный код, без Maven, исходников и промежуточных артефактов. Это уменьшает размер образа и ускоряет его загрузку [1].

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		40

Для сборки конкретного сервиса используется плагин `spring-boot-maven-plugin`, который формирует JAR-файл со встроенным сервером. В результате каждый сервис представляет собой самодостаточный исполняемый модуль, не требующий установки внешнего сервера приложений. Все зависимости, включая Tomcat, уже включены в JAR.

Оркестрация контейнеров описана в файле `docker-compose.yml`. Этот файл задаёт состав системы, связи между компонентами и их конфигурацию. В него включены все восемь сервисов системы: `config-server`, `eureka-server` и `api-gateway` как инфраструктурная основа, а также `auth-service`, `driver-service`, `passenger-service`, `ride-service` и `rating-service` как бизнес-логика. Для каждого сервиса указан свой Dockerfile, порт, переменные окружения и политика перезапуска.

Помимо сервисов, `docker-compose.yml` описывает вспомогательные контейнеры. Для каждой бизнес-БД поднимается отдельный экземпляр PostgreSQL. Всего их четыре: `driver_db`, `passenger_db`, `ride_db` и `rating_db`. Для каждого контейнера заданы имя БД, пользователь и пароль через переменные окружения `POSTGRES_DB`, `POSTGRES_USER` и `POSTGRES_PASSWORD`. Бизнес-сервисы подключаются к своей БД, используя эти учётные данные, переданные в конфигурации сервиса.

Контейнер Redis используется для кэширования геоданных в сервисе поездок. Контейнер Kafka работает в режиме KRaft, не требующем отдельного ZooKeeper. Kafka UI предоставляет веб-интерфейс для просмотра топиков, сообщений и потребителей. Контейнер Keycloak обеспечивает аутентификацию и авторизацию, его БД также развёрнута в отдельном контейнере PostgreSQL. PGAdmin предоставляет графический интерфейс для управления всеми БД.

Для локальной разработки предусмотрен упрощённый файл `docker-compose-local.yml`. Он поднимает только инфраструктурные компоненты - Redis, Kafka, Keycloak и БД, - но не запускает сами бизнес-сервисы. Разработчик запускает сервисы локально в IDE, а они подключаются к инфраструктуре из контейнеров. Это ускоряет цикл разработки, так как не требуется пересобирать Docker-образы после каждого изменения кода.

Конфигурационные настройки для всех окружений вынесены в файл `.env` в директории `docker`. В нём определены переменные: пароли для БД, учётные данные администратора Keycloak, API-ключ Marbox, адреса Kafka-брокеров и флаги включения мониторинга. Файл `.env` подключается в `docker-compose.yml`, и его значения подставляются в переменные окружения контейнеров. При развёртывании на новом хосте достаточно изменить значения в `.env` без правки `compose`-файла.

Для удобства управления в корне проекта создан Makefile. Команда `make up` запускает полное окружение описанное в `docker-compose.yml`. Команда `make down` останавливает и удаляет все контейнеры. Команда `make up-local` запускает облегчённое окружение из `docker-compose-local.yml`. Такой подход избавляет от необходимости запоминать длинные `docker compose` команды с путями к файлам.

Благодаря Eureka, сервисы автоматически находят друг друга при запуске. После того как `eureka-server` поднялся и принял регистрации от остальных сервисов, `api-gateway` начинает маршрутизировать запросы, а Feign-клиенты находят целевые сервисы по именам. Никакой ручной настройки адресов не требуется - вся маршрутизация строится динамически на основе данных из реестра сервисов [1].

5.3 Настройка CI/CD процессов и мониторинг серверной части

После развёртывания системы необходимо обеспечить наблюдение за её состоянием. В распределённой микросервисной архитектуре, где запрос пользователя может пройти через несколько сервисов, традиционные подходы к мониторингу недостаточны - требуется видеть не только факт ошибки, но и её причину в цепочке вызовов. Для решения этих задач в проекте используется стек технологий наблюдаемости, включающий сбор метрик, трассировку запросов и агрегацию логов [2].

Сбор метрик реализован с помощью Prometheus. Каждый микросервис предоставляет HTTP-эндпоинт `/actuator/prometheus`, отдающий метрики в формате, понятном Prometheus. Для включения этого эндпоинта в каждом сервисе добавлены зависимости `spring-boot-starter-actuator` и `micrometer-registry-prometheus`. Prometheus периодически опрашивает все сервисы и сохраняет полученные метрики в своей БД временных рядов. Среди собираемых метрик: количество HTTP-запросов (с разбивкой по методам и кодам ответа), время обработки запросов в различных перцентилях, использование памяти JVM, загрузка процессора, количество активных соединений с БД, статистика по Kafka-сообщениям [2].

Визуализация метрик выполняется в Grafana. Для проекта подготовлены дашборды, отображающие ключевые показатели каждого сервиса: количество запросов в секунду, долю ошибочных ответов, среднее и максимальное время ответа. Отдельный дашборд показывает состояние JVM: использование heap-памяти, активность сборщика мусора, количество потоков. Дашборд для Kafka отображает количество сообщений в топиках, задержку доставки и статистику потребителей. Все дашборды обновляются в реальном времени, что позволяет оперативно обнаруживать аномалии в работе системы [2].

Для распределённой трассировки запросов используется Tempo. Когда пользовательский запрос приходит на API Gateway, ему присваивается уникальный `trace_id`. Этот идентификатор передаётся всем нижестоящим сервисам через HTTP-заголовки, а при отправке сообщений в Kafka сохраняется в полях `trace_id` и `span_id` таблицы `kafka_messages` и восстанавливается на стороне потребителя. Благодаря этому в Tempo можно проследить полный путь запроса: какие сервисы были задействованы, сколько времени заняла обработка на каждом из них, на каком именно шаге произошла ошибка. Трассировка особенно полезна при отладке проблем с производительностью, когда нужно найти узкое место в длинной цепочке вызовов [2,5].

Экспорт трассировок в Tempo осуществляется через протокол OTLP (OpenTelemetry Protocol). В каждом сервисе добавлены зависимости для OpenTelemetry, а в конфигурации указан адрес Tempo-коллектора. Spring Boot автоматически инструментирует входящие и исходящие HTTP-запросы, добавляя к ним `trace`-заголовки. Для Kafka-сообщений инструментирование реализовано вручную в коде отправителя и получателя, что позволяет сохранять сквозную трассировку даже при асинхронном взаимодействии.

Агрегация логов выполняется с помощью связки Loki и Promtail. Promtail запускается как sidecar-контейнер и отслеживает лог-файлы всех сервисов. Он собирает записи, добавляет к ним метки с именем сервиса, и отправляет в Loki. Loki хранит логи и предоставляет API для их поиска. В Grafana логи отображаются рядом с метриками и трассировками, что даёт полную картину работы системы в одном интерфейсе. Например, при обнаружении всплеска ошибок через метрики Prometheus можно сразу перейти к логам со-

ответствующего сервиса и найти точное исключение, а затем по `trace_id` проследить весь путь запроса в Tempo [2].

Для управления версиями БД в процессе разработки и развёртывания используется Liquibase. Миграции хранятся в XML-файлах и описывают изменения схемы для каждой версии продукта. При старте сервиса Liquibase автоматически проверяет, какие миграции уже применены, и выполняет только новые. Это гарантирует, что схема БД всегда соответствует версии кода, как на локальной машине разработчика, так и в продуктивном окружении [6].

Процесс непрерывной интеграции строится вокруг Maven. При каждом изменении кода выполняется команда `mvn clean verify`, которая последовательно запускает компиляцию всех модулей, модульные тесты, интеграционные тесты и проверку Checkstyle. Если любой из этапов завершается неудачно, сборка считается проваленной, и разработчик получает уведомление. Такой подход гарантирует, что в основную ветку проекта попадает только код, прошедший полную автоматическую проверку [3].

Отдельного упоминания заслуживают механизмы отказоустойчивости, также являющиеся частью эксплуатации системы. Это ShedLock для предотвращения дублирования Kafka-сообщений при запуске нескольких экземпляров `ride-service`, Resilience4J для защиты от каскадных отказов при межсервисных вызовах и Outbox-паттерн для гарантированной доставки событий. Все эти механизмы были подробно рассмотрены в главе 4 при описании реализации соответствующих модулей [1,8].

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		43

6. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

6.1 Исходные данные для расчета экономического эффекта

Любая разработка программного обеспечения, приложения или их модулей требует финансовых и человеческих ресурсов.

Задачей данного дипломного проекта является разработка микросервисной платформы для заказа такси на языке программирования Java.

Разработка программного продукта предусматривает проведение всех стадий проектирования и относится к первой группе сложности.

Последовательность расчетов:

- Расчёт объёма функций программных модулей.
- Расчёт полной себестоимости микросервисной платформы заказа такси.
- Расчёт цены и прибыли по программному продукту.

6.2 Расчет объёма функций программного обеспечения

Общий объём ПО определяется по формуле (6.1), исходя из объёма функций, реализуемых программой.

$$V_0 = \sum_{i=0}^n V_i \quad (6.1)$$

где V_0 – общий объём ПО;

V_i – объём функций ПО;

n – общее число функций.

В том случае, когда на стадии технико-экономического обоснования проекта невозможно рассчитать точный объём функций, то данный объём может быть получен на основании прогнозируемой оценки имеющихся фактических данных по аналогичным проектам, выполненным ранее, или применением нормативов по каталогу функций.

По каталогу функций на основании функций разрабатываемых микросервисных модулей определяется общий объём. Также на основе зависимостей от организационных и технологических условий, был скорректирован объём на основе экспертных оценок.

Уточнённый объём ПО определяется по формуле (6.2).

$$V_y = \sum_{i=0}^n V_{yi} \quad (6.2)$$

где V_y – уточнённый объём ПО;

V_{yi} – уточнённый объём отдельной функции в строках исходного кода.

Перечень и объём функций микросервисных модулей приведён в таблице 6.1.

Таблица 6.1 – Перечень и объем функций программного обеспечения

					ДП.АС64.220047-05 81 00	Лист
						44
Изм.	Лист.	№ докум.	Подп.	Дата		

Номер функции	Наименование (содержание) функции	Объём функции строк исходного кода	
		По каталогу V_y	Уточнённый V_{yi}
101	Регистрация и аутентификация	450	520
102	Управление профилями водителей и пассажиров	350	420
103	Управление автомобилями (CRUD)	280	340
301	Создание и обработка заказов (ride-service)	1100	1410
302	Интеграция картографического сервиса и кэширование	380	490
401	Интеграция с брокером сообщений и Outbox	600	740
501	API-шлюз: маршрутизация	320	380
502	Межсервисная авторизация (JWT)	290	350
601	Сервис рейтинга и оценок	240	310
701	Service Discovery, конфигурация, Docker-контейнеризация	380	460
702	Мониторинг и тестирование (метрики, трассировка, логи)	900	1010
	ИТОГО	5290	6430

Учитывая информацию, указанную в таблице 6.1, о функциях разрабатываемого программного обеспечения, уточненный объем ПО (V_{yi}) составил 6430 строк исходного кода вместо предполагаемого количества строк 5290.

6.3 Расчет полной себестоимости программного продукта

Стоимостная оценка программного средства у разработчика предполагает составление сметы затрат, которая включает следующие статьи расходов:

- заработную плату исполнителей (основную – ЗПо и дополнительную – ЗПд);
- отчисления на социальные нужды ($P_{соц}$);
- материалы и комплектующие изделия (P_m);
- спецоборудование (P_c);
- машинное время ($P_{мв}$);
- расходы на научные командировки ($P_{нк}$);
- прочие прямые расходы ($P_{пр}$);
- накладные расходы ($P_{нр}$);
- затраты на освоение и сопровождение программного средства (P_o и $P_{со}$).

Полная себестоимость (C_n) разработки ПП рассчитывается как сумма расходов по всем статьям с учетом рыночной стоимости аналогичных продуктов.

Основной статьёй расходов на создание программного продукта является заработная плата проекта (основная и дополнительная) разработчиков (исполнителей) ($ЗП_{осн} + ЗП_{доп}$), в число которых принято включать инженеров-программистов, руководителей

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист	№ докум.	Подп.	Дата		45

проекта, системных архитекторов, дизайнеров, разработчиков баз данных, Web-мастеров и других специалистов, необходимых для решения специальных задач в команде.

Расчёт заработной платы разработчиков ПП начинается с определения:

- Продолжительности времени разработки ($\Phi_{рв}$), которое устанавливается экспертным путем с учётом сложности, новизны ПП и фактически затраченного времени. В данном дипломном проекте $\Phi_{рв} = 31$ день.

- Количества разработчиков программного обеспечения. В данном дипломном проекте один разработчик категории инженер-программист разряда 8. Заработная плата разработчиков определяется как сумма основной и дополнительной заработной платы всех исполнителей.

Основная заработная плата $ЗП_{осн}$ каждого исполнителя определяется по формуле (6.3):

$$ЗП_{осн} = T_{ст1р} \cdot \frac{T_k}{\Phi_{эфф.р.в.}} \cdot \Phi_{рв} \cdot K_{пр} \quad (6.3)$$

где $T_{ст1р}$ – размер базовой ставки, на текущий момент актуальное значение которой 250 бел.руб.;

T_k – тарифный коэффициент согласно разряду исполнителя;

$\Phi_{эфф.р.в.}$ – среднее количество рабочих дней;

$\Phi_{рв}$ – фонд рабочего времени исполнителя (продолжительность разработки программного модуля), дни;

$K_{пр}$ – коэффициент премии, $K_{пр} = 1,4$.

Тарифный коэффициент согласно 8 разряда инженера-программиста $T_k = 1,57$. Продолжительность разработки программного продукта – 70 дней, количество рабочих дней в месяце на текущий год составляет 22 дня. Рассчитаем основную заработную плату инженера-программиста, согласно формуле (6.3):

$$ЗП_{осн} = 250 \cdot \frac{1,57}{22} \cdot 70 \cdot 1,4 = 1748,41 \text{ (бел.руб.)}$$

Дополнительная заработная плата $ЗП_{доп}$ каждого исполнителя рассчитывается от основной заработной платы по формуле (6.4).

$$ЗП_{доп} = ЗП_{осн} \cdot \frac{H_{доп.зп}}{100\%} \quad (6.4)$$

где $H_{доп.зп}$ – надбавка дополнительной заработной платы, равная 15%.

Рассчитаем дополнительную заработную плату инженера-программиста, согласно формуле (4.4):

Рассчитаем заработную плату исполнителей проекта и результаты занесем в таблицу (6.4):

$$ЗП_{доп} = 1748,41 \cdot \frac{15\%}{100\%} = 262,26 \text{ (бел.руб.)}$$

Результаты вычислений внесём в таблицу 6.2.

Таблица 6.2 – Расчет заработной платы

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист.	№ докум.	Подп.	Дата		46

Категория работников	Разряд	Т _к	Φ _{эфф.р.в.}	К _{пр}	Зарботная плата, бел.руб.		
					Осн.	Доп.	Всего
Инженер-программист	8	1,57	70	1,4	1748,41	262,26	2010,67
Итого	—	—	—	—	1748,41	262,26	2010,67

Таким образом, как видно из таблицы, заработная плата инженера-программиста составляет 2010,67 бел. руб.

Отчисления на социальные нужды $P_{\text{соц}}$ определяются по формуле (6.5) в соответствии с действующим законодательством по нормативу.

$$P_{\text{соц}} = (ЗП_{\text{осн}} + ЗП_{\text{доп}}) \cdot \frac{35\%}{100\%} \quad (6.5)$$

Рассчитанное отчисления на социальные нужды:

$$P_{\text{соц}} = (1748,41 + 262,26) \cdot \frac{35\%}{100\%} = 703,73 \text{ (бел.руб.)}$$

Расходы на материалы и комплектующие $P_{\text{м}}$ отражают расходы на магнитные носители, бумагу, красящие ленты и другие материалы, необходимые для разработки программного продукта. Норма расхода материалов в суммарном выражении определяется в процентах к основной заработной плате (6.6).

$$P_{\text{м}} = ЗП_{\text{осн}} \cdot \frac{H_{\text{мз}}}{100\%} \quad (6.6)$$

где $H_{\text{мз}}$ – норма расхода материалов от основной заработной платы, в процентах.

Рассчитаем расходы на материалы и комплектующий, приняв $H_{\text{мз}}$ равным 4%:

$$P_{\text{м}} = 1748,41 \cdot \frac{4\%}{100\%} = 69,94 \text{ (бел.руб.)}$$

Расходы на спецоборудование $P_{\text{с}}$ включают затраты на приобретение технических и программных средств специального назначения, необходимых для разработки методического пособия, включая расходы на проектирование, изготовление, отладку и другое.

В данном дипломном проекте для разработки серверной части приложения приобретение какого-либо спецоборудования не предусматривалось. Так как спецоборудование не было приобретено, расходы равны нулю.

Расходы на машинное время $P_{\text{мв}}$ включают оплату машинного времени, необходимого для разработки и отладки программного продукта. Они определяются в машино-часах по нормативам на 100 строк исходного кода машинного времени. $P_{\text{мв}}$ определяется по формуле (6.7).

$$P_{\text{мвi}} = Ц_{\text{ми}} \cdot \frac{V_0}{100} \cdot H_{\text{мв}} \quad (6.7)$$

где $Ц_{\text{ми}}$ – цена одного машинного часа (7 бел. руб.);

V_0 – уточнённый общий объём машинного кода;

$H_{\text{мв}}$ – норматив расхода машинного времени на отладку 100 строк кода в машино-часах. Принимается в размере 0,7.

Рассчитаем расходы на машинное время:

$$P_{\text{мвi}} = 0,7 \cdot \frac{6430}{100} \cdot 0,6 = 270,06 \text{ (бел. руб.)}$$

Расходы по статье "Научные командировки" $P_{нк}$ берутся либо по смете научных командировок, разрабатываемой на предприятии, либо в процентах от основной заработной платы исполнителей (10-15%). Так как в данном проекте научные командировки не предусмотрены, данная статья не рассчитывается.

Прочие затраты $P_{пр}$ включают затраты на приобретение специальной научно-технической информации и специальной литературы. Определяются по нормативу в процентах к основной заработной плате исполнителей. Так как специальная научно-техническая информация и специальная литература не приобреталась, то данная статья не рассчитывается.

Затраты на накладные расходы $P_{нр}$ связаны с содержанием вспомогательных хозяйств, опытных производств, а также с расходами на общехозяйственные нужды. Определяется по нормативу в процентах к основной заработной плате по формуле (6.8).

$$P_{нр} = \frac{N_{нр}}{100\%} \cdot ЗП_{осн} \quad (6.8)$$

где $N_{нр}$ – норматив накладных расходов.

В данном дипломном проекте норматив накладных расходов равен 75%, поэтому затраты на накладные расходы согласно формуле (6.8) равны:

$$P_{нр} = \frac{75\%}{100\%} \cdot 1748,63 = 1311,31 \text{ (бел.руб.)}.$$

Сумма вышеперечисленных расходов по статьям на программный продукт служит исходной базой для расчёта затрат на освоение и сопровождение программного продукта. Они рассчитываются по формуле (6.9):

$$СЗ = ЗП_{осн} + ЗП_{доп} + P_{соц} + P_{м} + P_{с} + P_{мв} + P_{нк} + P_{пр} + P_{нр} \quad (6.9)$$

тогда

$$СЗ = 1748,41 + 262,26 + 703,73 + 69,94 + 270,06 + 1311,31 = 4365,71 \text{ (бел.руб.)}.$$

Организация-разработчик участвует в освоении программного продукта и несёт соответствующие затраты, на которые составляется смета, оплачиваемая заказчиком по договору. Затраты на освоение P_o определяются по установленному нормативу от суммы затрат по формуле (6.10).

$$P_o = СЗ \cdot \frac{N_o}{100\%} \quad (6.10)$$

где $СЗ$ – сумма вышеперечисленных расходов по статьям на разработку программного продукта;

N_o – установленный норматив затрат на освоение. Для данного дипломного проекта принимается равным 8%.

Рассчитаем затраты на освоение продукта:

$$P_o = 4365,71 \cdot \frac{8\%}{100\%} = 349,26 \text{ (бел.руб.)}$$

Организация-разработчик осуществляет сопровождение программного продукта и несёт расходы, которые оплачиваются заказчиком в соответствии с договором и сметой на сопровождение. Расходы на сопровождение $P_{со}$ рассчитываются по формуле (6.11).

$$P_{co} = C3 \cdot \frac{H_{co}}{100\%} \quad (6.11)$$

где H_{co} – установленный норматив затрат на сопровождение программного продукта. Для данного дипломного проекта принимается равным 7%.

Рассчитаем расходы на сопровождение:

$$P_{co} = 4365,71 \cdot \frac{7\%}{100\%} = 305,60 \text{ (бел.руб.)}$$

Полная себестоимость (СП) разработки программного продукта рассчитывается как сумма расходов по всем статьям. Она определяется по формуле (6.12).

$$СП = C3 + P_o + P_{co} \quad (6.12)$$

Рассчитанная полная себестоимость продукта:

$$СП = 4365,71 + 349,26 + 305,60 = 5020,27 \text{ (бел.руб.)}$$

Результаты вычислений занесём в таблицу 6.3.

Таблица 6.3 – Себестоимость программного продукта

Наименование статей затрат	Норматив, %	Сумма затрат, бел.руб.
Заработная плата, всего	–	2010,67
Основная заработная плата	–	1748,41
Дополнительная заработная плата	–	262,26
Отчисления на социальные нужды	35	703,73
Спецоборудование	Не применялось	–
Материалы и комплектующие изделия	4	69,94
Машинное время	–	270,06
Научные командировки	Не планировались	–
Прочие затраты	Не применялись	–
Накладные расходы	75	1311,31
Сумма затрат	–	4365,71
Затраты на освоение	8	349,26
Затраты на сопровождение	7	305,60
Полная себестоимость	–	5020,57

В результате всех расчётов полная себестоимость программного продукта составила 5020,57 бел.руб.

6.4 Расчет цены и прибыли по программному продукту

Для определения цены программного продукта необходимо рассчитать плановую прибыль Π , которая рассчитывается по формуле (6.13).

$$\Pi = СП \cdot \frac{R}{100\%} \quad (6.13)$$

где СП – полная себестоимость программного модуля, бел. руб;

R – уровень рентабельности программного модуля.

Рассчитаем прибыль от реализации:

$$П = 5020,57 \cdot \frac{23\%}{100\%} = 1154,73 \text{ (бел.руб)}$$

После расчета прибыли от реализации по формуле (6.14) определяется прогнозируемая цена программного продукта без налогов $Ц_{п}$.

$$Ц_{п} = СП + П \quad (6.14)$$

Рассчитаем цену программного продукта без налогов:

$$Ц_{п} = 5020,57 + 1154,74 = 6175,3 \text{ (бел.руб.)}$$

Отпускная цена $Ц_0$ (цена реализации) программного продукта включает налог на добавленную стоимость и рассчитывается по формуле (6.15).

$$Ц_0 = СП + П + НДС_{пп} \quad (6.15)$$

где НДС_{пп} – налог на добавленную стоимость для программного продукта.

Для данного программного продукта НДС_{пп} рассчитывается по формуле (6.16).

$$НДС_{пп} = Ц_{п} + \frac{НДС}{100\%} \quad (6.16)$$

где НДС – налог на добавленную стоимость. В настоящее время он составляет 20%.

Рассчитаем НДС_{пп}:

$$НДС_{пп} = 6175,30 + \frac{20\%}{100\%} = 1235,06 \text{ (бел.руб.)}$$

Рассчитаем НДС_{пп}:

$$Ц_0 = 5020,57 + 1154,73 + 1235,06 = 7410,36 \text{ (бел.руб.)}$$

Прибыль от реализации программного продукта за вычетом налога на прибыль является чистой прибылью. Чистая прибыль остаётся организации-разработчику и представляет собой экономический эффект от создания нового программного продукта. Она рассчитывается по формуле (6.17).

$$ПЧ = П \cdot (1 - \frac{Н_{п}}{100\%}) \quad (6.17)$$

где $Н_{п}$ – ставка налога на прибыль. В настоящее время он равен 20%.

Рассчитаем чистую прибыль:

$$ПЧ = 1154,73 \cdot (1 - \frac{20\%}{100\%}) = 923,78 \text{ бел.руб.}$$

Результаты расчётов цены и прибыли по программному продукту сведены в таблицу 6.4.

Таблица 6.4 – Себестоимость программного продукта

					ДП.АС64.220047-05 81 00	Лист
						50
Изм.	Лист.	№ докум.	Подп.	Дата		

Наименование статей затрат	Норматив, %	Сумма затрат, бел.руб.
Полная себестоимость	–	5020,57
Прибыль	20	1154,73
Цена без НДС	–	6175,30
НДС	20	1235,06
Отпускная цена	–	7410,36
Налог на прибыль	20	230,95
Чистая прибыль	–	923,78

В ходе произведенных расчетов определены основные экономические показатели: полная себестоимость – 5020,57 бел.руб.; прогнозируемая цена – 7410,36 бел. руб.; чистая прибыль – 923,78 бел.руб.

Разработанный программный продукт ориентирован на рынок такси-сервисов, где конкуренция достаточно высока. При этом стоимость его разработки остаётся на приемлемом уровне для стартапа или малого предприятия. Стоимость системы, полученная в ходе расчётов, является конкурентной по сравнению с готовыми решениями, представленными на рынке. Основным преимуществом разработанной платформы является возможность гибкого расширения функциональности за счёт микросервисной архитектуры, а также кроссплатформенность за счёт контейнеризации. Таким образом, рассчитанная отпускная цена на серверную часть приложения, разрабатываемую в рамках данного дипломного проекта, составляет 7410,36 белорусских рубля и является конкурентоспособной. При расчете цены учтены отчисления в фонд социальной защиты, а также налоги, необходимые к уплате.

ЗАКЛЮЧЕНИЕ

В рамках данного дипломного проекта была разработана серверная часть приложения заказа такси на основе микросервисной архитектуры с использованием языка Java и экосистемы Spring Boot.

В процессе выполнения дипломного проекта были решены следующие задачи:

1. Проведён анализ предметной области сервисов заказа такси, выделены основные бизнес-сущности и процессы.
2. Выполнен сравнительный анализ монолитной и микросервисной архитектур, обоснован выбор микросервисного подхода.
3. Спроектирована архитектура системы: схема взаимодействия микросервисов, структура БД, REST API конечных точек.
4. Реализованы ключевые модули: сервис поездок с конечным автоматом, геолокация с кэшированием, авторизация через JWT, асинхронное взаимодействие через Kafka.
5. Разработаны модульные и интеграционные тесты с использованием TestContainers, Cucumber и REST Assured.
6. Выполнена контейнеризация системы с помощью Docker, настроен мониторинг на базе Prometheus, Grafana, Tempo и Loki.
7. Проведён расчёт технико-экономической эффективности, определена себестоимость и отпускная цена продукта.

Разработанная система удовлетворяет следующим ключевым требованиям:

1. Микросервисная архитектура с независимым хранением данных для каждого сервиса.
2. Синхронное взаимодействие через REST API и асинхронное - через брокер сообщений Kafka.
3. Аутентификация и авторизация на основе JWT с ролевой моделью доступа.
4. Кэширование часто запрашиваемых данных с использованием Redis.
5. Контейнеризация и мониторинг для обеспечения отказоустойчивости и наблюдаемости.

Проведённое тестирование подтвердило корректность реализации ключевых бизнес-сценариев: регистрация пользователей, создание заказа, назначение водителя, обработка переходов между статусами поездки и обмен событиями через Kafka.

Для разработанного программного обеспечения выполнен расчёт экономической эффективности: полная себестоимость составила 5020,57 белорусских рублей, отпускная цена - 7410,36 белорусских рубля, чистая прибыль - 923,78 белорусских рубля. Полученные значения подтверждают экономическую целесообразность разработки.

					ДП.АС64.220047-05 81 00	Лист
Изм.	Лист.	№ докум.	Подп.	Дата		52

СПИСОК СОКРАЩЕНИЙ

БД	–	база данных.
ИС	–	информационная система.
ПО	–	программное обеспечение.
ПП	–	программный продукт.
СУБД	–	система управления базами данных.
ТЗ	–	техническое задание.
API	–	(Application Programming Interface) программный интерфейс приложения.
HTTP	–	(HyperText Transfer Protocol) протокол передачи гипертекста.
IDE	–	(Integrated Development Environment) интегрированная среда разработки.
JDK	–	(Java Development Kit) комплект разработчика приложений на языке Java.
JSON	–	(JavaScript Object Notation) текстовый формат обмена данными.
JPA	–	(Java Persistence API) спецификация Java EE, описывающая систему управления сохранением объектов.
JWT	–	(JSON Web Token) стандарт для создания токенов доступа, используемый для авторизации.
ORM	–	(Object-Relational Mapping) технология программирования, которая связывает базы данных с концепциями объектно-ориентированных языков программирования.
REST	–	(Representational State Transfer) архитектурный стиль взаимодействия компонентов распределённого приложения.
SQL	–	(Structured Query Language) язык структурированных запросов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ричардсон, К. Микросервисы. Паттерны разработки и рефакторинга / К. Ричардсон ; пер. с англ. А. Слинкина. – СПб. : Питер, 2020. – 544 с.
2. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман ; пер. с англ. Н. Вильчинского. – СПб. : Питер, 2023. – 688 с.
3. Карнелл, Дж. Микросервисы Spring в действии / Дж. Карнелл, И. Санчес ; пер. с англ. – 2-е изд. – СПб. : Питер, 2022. – 448 с.
4. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин ; пер. с англ. А. А. Слинкина. – СПб. : Питер, 2021. – 352 с.
5. Spring Boot Reference Documentation [Электронный ресурс] / VMware, Inc. – 2026. – Режим доступа: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. – Дата доступа: 19.04.2026.
6. PostgreSQL 16 Documentation [Электронный ресурс] / PostgreSQL Global Development Group. – 2025. – Режим доступа: <https://www.postgresql.org/docs/16/index.html>. – Дата доступа: 19.04.2026.
7. Redis Geospatial [Электронный ресурс] / Redis Ltd. – 2026. – Режим доступа: <http://redis.io/docs/latest/develop/data-types/geospatial/>. – Дата доступа: 19.04.2026.
8. Apache Kafka Documentation [Электронный ресурс] / The Apache Software Foundation. – 2026. – Режим доступа: <https://kafka.apache.org/documentation/>. – Дата доступа: 19.04.2026.
9. СТБ ISO/IEC 12207-2011. Информационные технологии. Процессы жизненного цикла программных средств. – Минск : Госстандарт, 2011. – 112 с.
10. ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов. – М. : Стандартиформ, 2010. – 3 с.